

Towards maintainable cross-platform software development through an object-oriented data access API and ODM tool for Firebase



Supervised by Prof. Nearchos Paspallis

A thesis presented in partial fulfilment of the requirements of

University of Central Lancashire

for the degree of

Master of Science in Computing

Nicos Kasenides

July 2025

Declaration

This dissertation is the result of my own work and includes nothing, which is the outcome of work done in collaboration except where specifically indicated in the text. It has not been previously submitted, in part, or, as a whole, to any university or institution for any degree, diploma, or other qualification. I have considered the information about plagiarism, as presented in the University Academic Regulations and the Assessment Handbook.

Signed, Nicos Kasenides

A stylized, handwritten signature in black ink that reads "Kasenides". The letter 'K' is large and ornate, with a long vertical stroke that curves at the bottom. The rest of the name is written in a cursive, flowing script.

Date: 17/07/2025

University of Central Lancashire, Cyprus

Contents

List of Figures	viii
List of Tables	x
1 Introduction	2
1.1 Background and Context	2
1.2 Objectives	5
1.3 Research Questions	6
1.4 Research Approach Overview	7
1.5 Thesis Structure	8
2 Related work	9
2.1 Introduction	9
2.2 NoSQL Datastores	9
2.2.1 Key-Value datastores	10
2.2.2 Document datastores	10
2.2.3 Column-family datastores	11
2.2.4 Graph datastores	11
2.2.5 Other characteristics	12
2.3 Firebase	12
2.4 ODM tools	13

2.5	Challenges	14
2.6	Impact on software development	15
2.7	Summary	16
3	Planning and Methodology	17
3.1	Introduction	17
3.2	Research design	17
3.3	Development Methodology & Deliverables	19
3.3.1	Development methodology	19
3.3.2	Deliverables & Timeline	19
3.4	Requirements specification	20
3.4.1	Functional requirements	21
3.4.2	Non-functional Requirements	23
3.5	Tools & Technologies	24
3.6	Risk management	25
3.7	Evaluation strategy	26
3.7.1	Performance	26
3.7.2	Maintainability	27
3.7.3	Feature coverage comparative analysis	28
3.8	Summary	29
4	Design	30
4.1	Introduction	30
4.2	Overview	30
4.3	Software Architecture components	32
4.4	Data modeling	33
4.4.1	Data types	33
4.4.2	Data organization and constraints	33

4.5	Validation and generation	36
4.5.1	Entity validation	36
4.5.2	Code generation	38
4.5.3	Data access API	39
4.6	Summary	41
5	Implementation	44
5.1	Package structure	44
5.2	Model definitions & Validation	45
5.2.1	Defining data entities	45
5.2.2	Data entity validation	46
5.3	Code generation	49
5.4	Delegate implementation	53
5.4.1	CRUD operations	55
5.4.2	Listing and Queries	61
5.4.3	Advanced features	61
5.5	Demonstrator app	63
5.6	Deployment & Guide	65
5.7	Summary	70
6	Testing	71
6.1	Introduction	71
6.2	Testing objectives	71
6.3	Testing platform	72
6.4	Unit Testing	72
6.5	Integration Testing	73
6.6	Compatibility Testing	76
6.7	Summary	76

7	Evaluation	77
7.1	Introduction	77
7.2	Performance overheads	77
7.2.1	Measurement procedure	78
7.2.2	Results	79
7.2.3	Discussion	84
7.3	Code maintainability	87
7.3.1	Measurement procedure	87
7.3.2	Results	88
7.3.3	Discussion	90
7.4	Comparative feature analysis	91
7.4.1	Procedure	91
7.4.2	Results	92
7.4.3	Discussion	94
7.5	Design strategies	95
7.6	Summary	96
8	Conclusion	98
8.1	Contributions	98
8.1.1	Objectives	98
8.1.2	Software requirements coverage assessment	100
8.1.3	Summary of research output	103
8.2	Limitations and future work	104
8.2.1	Development and Testing	104
8.2.2	Research approach	105
8.3	Epilogue	107
A	Unit test cases	108

B	Integration test cases	114
C	Feature matrix resources	117
C.1	Mongoose	117
C.2	Typegoose	117
C.3	Sequelize	118
C.4	Objectify	118
C.5	Beanie	118
C.6	MongoEngine	119
C.7	Spring Data MongoDB	119
C.8	JNoSQL	119
C.9	MongoDB .NET Driver	120
C.10	Mongoid	120
C.11	Marshmallow	120
C.12	Objection.js	121
C.13	Morphia	121
C.14	Laravel MongoDB (Jenssegers)	121
C.15	Waterline (Sails.js ORM)	121
C.16	Pydantic ODM	121
C.17	Isar	122
C.18	Typesaurus	122
C.19	mgo/bson	122
C.20	mongo-go-driver	122
C.21	bongo	122
C.22	NoRM	123
C.23	Hibernate	123

D ODM Feature tables	124
D.1 Databases, type safety, schemas	124
D.2 Queries, asynchronous operations, hooks, and validation	125
D.3 Relationships, realtime listeners, and pagination	126
D.4 Aggregation, transactions, migration, and maintenance	127
Bibliography	128

List of Figures

3.1	A Gantt Chart showing the timeline of the project.	20
3.2	Maintainability Index formula.	28
4.1	Firestore setup and usage workflow	31
4.2	A visualization of the data entity validation pipeline.	36
4.3	An overview of the data access API, including its classes, functions, and objects. More details about each delegate can be seen in Figure 4.4	42
4.4	Functions within each delegate in the data access API.	43
5.1	A visualization of the delegate abstraction in Firestore.	54
5.2	Creating and saving a contact using Firestore in the demo app.	66
5.3	Updating a contact in the demo app.	67
5.4	Deleting a contact in the demo app.	68
7.1	Average processing times for single-object operations in Firestore Native API and Firestore	80
7.2	Average processing times for multi-object operations in Firestore Native API and Firestore	81
7.3	Average processing times for single-object operations in RDB Native API and Firestore	82
7.4	Average processing times for multi-object operations in RDB Native API and Firestore	83

7.5	Firestore: Count of operations per performance category.	85
7.6	Realtime database: Count of operations per performance category	86

List of Tables

3.1	A summary of the timeline and stages in the project.	18
3.2	Functional requirements, prioritized using MoSCoW.	21
3.3	Non-functional requirements, prioritized using MoSCoW.	23
3.4	Identified risks with impact, probability, and mitigation strategies.	25
4.1	Supported Dart data types in Firestore and Realtime database.	34
7.1	Comparison of Firestore Native API and Firestore performance across single object operations.	80
7.2	Comparison of Firestore Native API and Firestore performance for multi- object operations.	81
7.3	Comparison of Realtime database Native API and Firestore performance for single-object operations.	82
7.4	Comparison of RDB Native API and Firestore performance for multi-object operations.	83
7.5	Comparison of Maintainability Index for the Firestore API and Firestore operation.	89
7.6	Frequency of feature support across all tools.	93
7.7	Number of supported features per tool.	93
8.1	Assessment of implementation for functional requirements.	100
8.2	Assessment of implementation for non-functional requirements.	102

A.1	Unit Test Cases	109
B.1	Integration test cases	115
D.1	ODM tool features – Database, type safety, and schema enforcement	124
D.2	ODM tool features – Queries, Async support, Hooks, and Validation	125
D.3	ODM tool features – Relationships, Realtime listeners, and Pagination	126
D.4	ODM tool features – Aggregation, Transactions, Migration, and Maintenance	127

List of Listings

1	An example of how an Firestore-managed entity can be defined.	46
2	<code>FirestoreBuilder</code> code fragment, used to detect Dart code elements and store them for later use.	47
3	Running filtering operations using <code>ClassChecker.filter()</code>	48
4	<code>ClassChecker.filter()</code> , showing the filtering process.	48
5	<code>FSTypes.isTypeSupported()</code> code fragment demonstrating the use of recursion to analyze class hierarchies for valid attributes.	50
6	<code>FirestoreBuilder</code> fragment showing the generation of imports and extensions.	51
7	Generated <code>toMap()</code> method for the <code>Person</code> class.	52
8	Generated <code>fromMap()</code> method for the <code>Person</code> class.	52
9	The <code>FS.create.one()</code> method.	56
10	The <code>RDB.create.one()</code> method.	57
11	A fragment of the <code>FS.create.many()</code> method.	58
12	A fragment of the <code>RDB.create.many()</code> method.	58
13	The implementation of <code>FS.get.many()</code>	60
14	An example use of the <code>FS.list.filter()</code> method.	61
15	An example use of transactions in Firestore.	62
16	The implementation of <code>FS.listen.handleDocumentListener()</code>	64
17	An integration test, testing the behavior of <code>FS.paginate()</code>	74
18	An integration test, testing the behavior of <code>RDB.list.ofClass()</code>	75

19	An integration test, testing the behavior of object-oriented transactions in Firestore.	75
20	The pagination function used for MI analysis, in blocking mode, without error checking.	89

Abstract

The increasing diversity and complexity of modern software applications has led developers to adopt flexible Backend-as-a-Service (BaaS) solutions like Google’s Firebase. Commercially accessible Artificial Intelligence and the Internet of Things, among others, have contributed to increasing demands in implementing and deploying data-intensive distributed systems. These systems can utilize NoSQL datastores like Firestore and the Realtime database that can offer increased scalability, higher availability, and ease of integration in mobile applications.

Despite providing several advantages, such datastores lack mature Object-Document Mapping (ODM) tools in cross-platform frameworks such as Flutter, which are particularly useful for bridging the impedance mismatch between application code and database structures in document-based systems. This forces developers to create and maintain boilerplate code to handle various tasks such as data validation, serialization, and deserialization, which increases error rates, negatively impacts code maintainability, reduces productivity, and ultimately leads to an increased time to market.

This thesis presents the design, implementation, and evaluation of Firestorm, which is an object-oriented data access API (Application Programming Interface) and ODM tool for use with Firebase’s Firestore and Realtime database. Firestorm aims to remove the impedance mismatch between object-oriented code data models used in application logic and the schemaless, document-based structure used in NoSQL datastores by providing type-safe, declarative mechanisms with which developers can define data entities, generate serialization

and deserialization logic at compile-time, and utilize a unified set of data management operations under a common data access API.

This project follows the design science paradigm and combines traditional approaches with agile methods such as iterative and incremental development. The requirements specification derives several requirements from relevant academic literature as well as existing ORM (Object-Relational Mapping) and ODM tools. These aim to address community needs and the gaps between existing solutions. The design and implementation of Firestorm focuses on a layered architecture which decouples various components and processes such as data modelling and validation, code generation, and datastore operations. Several code design patterns and other principles such as Delegates, Facades, Registries, and Builders have been utilized, helping to underpin Firestorm’s extensible support for multiple datastores. Furthermore, solutions to important challenges such as the lack of Reflection within Flutter applications, support for multithreading, as well as the complex nature of code analysis and generation are explored within this thesis.

The evaluation of Firestorm focused on three aspects: measuring its performance overheads, the level of code maintainability, and its feature coverage. An evaluation platform was used to conduct performance benchmarking in a controlled environment. This compared several operations implemented using Firestorm’s API against operations implemented using the native Firebase SDKs (Software Development Kit) for Firestore and the Realtime database. The comparison revealed that Firestorm not only stays within an acceptable 10% performance overhead in terms of processing speed, but also surpasses code written using the native APIs by improving execution speed. The maintainability of code written using Firestorm’s data access API was measured using the Maintainability Index. This was compared against code written using the native APIs, showing an average improvement of $\approx 10\%$ across the sampled operations, with more pronounced benefits as the operation complexity increased. Furthermore, a comparative analysis of feature coverage compared Firestorm against 20 ODM and ORM tools based on 13 features, showing that Firestorm

has a very good level of feature coverage of approximately 77% and placing it among the most comprehensive tools of its type.

These findings suggest that Firestorm can meaningfully improve the level of code maintainability and Developer Experience (DX) in cross-platform applications without sacrificing performance. Thus, it can offer a robust alternative for developers to avoid manually implementing data mapping layers to interact with Firebase datastores in Flutter applications. Despite these benefits, the very strict timeline of 8 weeks limited the implementation of additional advanced features such as connected fields and aggregation queries. The same time constraints led to a limited test coverage and compatibility testing, which focused primarily on the Android ecosystem. These, along with several other limitations in research methodology such as the lack of recording and addressing developer feedback will be addressed in future work which will help to refine and evolve this tool. Despite these limitations, the findings of this thesis suggest that Firestorm can substantially contribute toward maintainable cross-platform application development.

Acknowledgements

First and foremost, I would like to thank my supervisor Professor Nearchos Paspallis, who has always been eager to provide support, guidance, and feedback not only for this dissertation but on countless other occasions during the last decade. His mentorship, encouragement and insights have played an instrumental role in my development over all these years, and for that I am incredibly thankful.

I would also like to express my appreciation to the Rector of UCLan Cyprus, Professor Irene Polycarpou and UCLan Cyprus as an institution for providing financial support in the form of a scholarship, which enabled me to focus on completing this course without worrying about financial constraints.

My amazing friend and colleague, Dr Andriani Piki, has helped me grow both professionally and personally through her encouragement and support in every step, and I am thankful for our extraordinary cooperation.

At a personal level, I am incredibly blessed to enjoy the unconditional love and support of my wife Panayiota. Her presence in my life has been an enduring source of love, strength, purpose, and motivation throughout my life, studies, and career. Without these, anything else would be meaningless.

I would also like to thank my parents, Andreas and Vicky, who have instilled scientific curiosity and the moral compass to navigate the many challenges of life from a young age, leading me to always pursue knowledge and understanding.

Last but not least, I want to thank **me**.

I want to thank me, for believing in me.

I want to thank me, for doing all this hard work.

I want to thank me, for having no days off.

I want to thank me, for never quitting.

I want to thank me, for always being a giver.

I want to thank me, for trying to do more right than wrong.

I want to thank me, for just being me at all times.

— Snoop Dogg

Chapter 1

Introduction

1.1 Background and Context

Software applications have grown increasingly complex over the years. The rise of commercially accessible Artificial Intelligence (AI) and the Internet of Things (IoT) means that the services provided by such apps must often carry data intensive operations (Yousuff et al., 2023; Yazid et al., 2021). There are a number of options to choose from to enable data persistence when developing software, and the choice usually depends on the software requirements for each product. Traditional database management involves the use of *Relational Database Management Systems* (RDBMS) such as MySQL, Microsoft SQL, or Oracle SQL, which are constructed based on the mathematical principles of sets and relations (Halvorsen, 2014). Such systems organize information using tables, rows/records, and columns/attributes, which have specific properties, such as strict schemas, typesystems, and leverage relationships to create dependencies between information (Maier, 1983). The use of RDBMSs in various types of software enables complex queries through the use of SQL (Structured Query Language), a high level of data integrity, and support for various types of security constraints.

Despite their prominence, RDBM systems are not inherently scalable, which impacts

their usefulness when it comes to managing large amounts of data such as those seen in Deep Learning (DL), or those used by context-sensing systems. For this reason, many systems choose to adopt NoSQL datastores which provide more flexibility by following a less rigid approach to structuring data. NoSQL datastores store information in key-value pairs, making them significantly easier to scale through *partitioning* and *sharding* (Liu et al., 2024; Shethiya, 2025; Ponnusamy and Gupta, 2024). This leads to more efficient scaling, and ultimately better performance especially under real-time constraints (Ravindran and George, 2018; Liao et al., 2019). For example, Firebase, a popular Backend-as-a-Service (BaaS) offering by Google, provides two types of NoSQL datastores: Firestore, and Realtime Database. Firestore falls under the category of document-based NoSQL datastores, whereas the Realtime Database falls under the category of key-value based datastores – akin to following a JSON structure (Kamal et al., 2019). Due to their increased flexibility and ease of use, such datastores are particularly suitable for cross-platform applications, such as those developed using the Flutter and React Native frameworks.

While there are plenty of advantages when using these datastores, lower adoption compared to RDBMSs means that there is a lack of support and tools, especially when it comes to combining and using these in cross-platform software. Cross-platform frameworks like Flutter and React Native are built using languages such as Dart and JavaScript, which are inherently object-oriented. This means that source code for this type of software, as with many other types of modern software, is centered around the object-oriented principles of inheritance, polymorphism, abstraction, and encapsulation (Mathiassen et al., 2000). Conversely, relational databases use tabular structures while NoSQL databases utilize key-value structures, which are significantly different from object-structured allocated memory. These types of data representations cannot be directly translated into memory/application objects that can be used in code directly, leading to an *impedance mismatch* between the two types of structures. While relational databases accommodate for this problem through the use of tried-and-tested *Object-Relational-Mapping* (ORM) techniques and tools, NoSQL data-

stores, and particularly those offered by Firebase, lack mature Object-Document Mapping (ODM) support, especially for cross-platform software written in Dart and Flutter. As a consequence, developers must write and maintain boilerplate code which enables the conversion to and from objects and key-value/document structures – increasing the maintenance cost of software, and introducing various types of bugs that are impervious to many hours of debugging. The result is poor code portability, limited reusability, and poor *Developer Experience* (DX) due to increasing overheads over time (Karanikiotis and Symeonidis, 2024).

Some tools have been developed to solve this problem in different environments. For instance, the Java and Node.js ecosystems offer ODM tools like Morphia and Sequelize to convert to and from documents and memory objects. Interestingly, an effort is underway to create ODM tools for cross-platform, Flutter-based environments as well. The Firebase team has been developing the `firestore_odm` package, which “*provides generated Cloud Firestore bindings for Dart classes, allowing type-safe queries and updates*” (pub.dev, 2025). While offering a significant step forward, this tool – like many other Flutter features – has seen relatively slow progress. The ever-present need for developers to efficiently manage data across cross-platform software means that this slow progress does not effectively capture the needs of developers for data management. Furthermore, this and various other ODM packages for Flutter applications have very poor documentation as witnessed in their example codes, and README documents, making them harder to understand and use. Finally, tools such as Firestore ODM reduce code maintainability by generating code which is not backward/forward compatible and does not enable the use of an extensible data access API.

This thesis aims to address these problems by designing, implementing, and evaluating *Firestorm*, a *data access API* (Application Programming Interface) and ODM tool for Flutter-based applications, which is specifically tailored towards the Firestore and Realtime Database. By leveraging various software engineering concepts such as model-driven design, it aims to solve the impedance mismatch problem, enabling developers to seamlessly manage data in these datastores without having to introduce additional logic in their software. As

established, such a tool can be an important asset for maintainable software development. This builds on top of previous work which focused on building such an ODM tool for Java-based environments. While there are similarities between Java and Dart, the realization of such a tool in Dart entails bridging various challenges, such as the lack of support for Reflection in Flutter apps, differences in language syntax and features, and variations in how languages support object-oriented design. Finally, a major challenge is the reduction of serialization/deserialization overheads, which are introduced during the document conversion process. Such overheads have been reduced in the Java-based environment by leveraging techniques including multithreading through Java’s Executor Framework. Meanwhile, Dart uses different constructs such as concurrent execution through its event loop and Isolates mechanisms which provide support for parallel execution. Ultimately, the evaluation of Firestore based the aspects of performance, code maintainability, and feature coverage, can lead to conclusions about the package’s usefulness and provide insights to its limitations.

1.2 Objectives

The first objective of this thesis is to identify and study relevant research works and similar tools which have been developed in the past, with the aim of uncovering key challenges and gaps, as well as identifying key features of ORM/ODM tools. The second objective is to design, develop, and test Firestore as a Flutter package. Finally, the tool will be evaluated with respect to its performance, code maintainability, and feature coverage. This will measure the tool’s usefulness as a development tool, and discover its limitations.

Firebase’s set of tools, including the Firestore and Realtime Database, are among the most popular options when it comes to adding backend support to cross-platform software. Such cross-platform products are used in various contexts and domains, including educational, commercial, and research projects. Yet, despite their prominence, the Dart/Flutter developer community lacks a reliable, developer-friendly ODM solution to support develop-

ment in both the Firestore and Realtime database. Based on this, the objectives of this tool are to:

1. Provide a consistent, declarative approach to model, access, and manipulate data by leveraging model-driven development practices.
2. Reduce or eliminate repetitive boilerplate code to manage data.
3. Facilitate a Rapid Application Development (RAD) approach by accelerating the development of software prototypes.
4. Improve code readability, maintainability, and portability, especially in larger-scale Flutter projects.
5. Support both datastores using a consistent data access API.

Such a tool may also provide a way to support research and teaching in software engineering by encouraging the use of object-oriented design, data abstraction practices, and the efficient use and management of data in applications requiring scalability, such as those frequently utilized in Machine Learning and Data Analytics.

1.3 Research Questions

This thesis will investigate the following research questions, which aim to explore both the design and effectiveness of Firestorm as a real-world development tool:

- **RQ1:** To what extent does using Firestorm improve code maintainability in developed applications compared to using native Firebase SDKs?
- **RQ2:** Which design principles and architectural strategies enable Firestorm to support both Firestore and the Realtime Database within a unified ODM framework?

- **RQ3:** What are the performance overheads introduced by Firestorm compared to the Firebase SDKs, in terms of processing time?
- **RQ4:** How does Firestorm compare against other ODM tools in terms of features?

It is hypothesized that Firestorm will slightly improve code maintainability compared to native Firebase SDKs, but at the sacrifice of added performance overheads. Furthermore, it is expected that Firestorm will support a decent number of features but not be ranked among the top tier tools which are already available for other types of programming environments.

1.4 Research Approach Overview

The first phase of the research methodology entails the design and development of Firestorm through an iterative development process. The output of this first phase is to construct an experimental version of the tool, mostly for proof-of-concept and evaluation purposes. This initial version will be integrated into a test platform which will enable the evaluation of the tool. The testing phase will focus on functional correctness by utilizing Flutter’s testing suite to conduct unit and integration testing.

Firestorm will be evaluated with respect to the research questions identified above, by using the testing platform. Firstly, Firestorm’s code maintainability will be measured and compared against that of using the traditional Firebase database SDKs. This will be done by measuring the maintainability of various snippets of code written using both approaches and determining their Maintainability Index (MI) (Heričko and Šumak, 2023). The second research question is more design-focused rather than empirical, and directly targets a major design objective of this thesis. To answer the question, architectural design choices will be enumerated to discuss their impact on enabling support for both datastores. To measure the performance overheads introduced by Firestorm, a set of database operations will be selected and implemented using Firestorm and the Firebase SDKs for each datastore. For each of these pieces of code, the processing overhead can be measured as the processing

time between an operation's start and finish. These will then be compared to determine the performance overheads of Firestorm. For the last research question, the features of Firestorm and other ODM tools such as Firestore ODM will be explored to conduct a comparative analysis.

1.5 Thesis Structure

The rest of this thesis is organized in chapters as follows. The next chapter (2) explores existing literature and tools which are relevant to NoSQL, Firebase, and ODM tools, such as their software architectures, the challenges faced, and their impact on software development. Chapter 3 outlines the project's timeline, specifies the software requirements, and discusses the research methodology. In chapter 4, the software architecture of Firestorm is discussed, which is followed by details regarding the development and testing of the tool in chapters 5 and 6. Chapter 7 presents and discusses the results of the evaluation with respect to the research questions. Finally, chapter 8 concludes the thesis by reflecting on the project's contributions and impact and suggesting potential directions for future work.

Chapter 2

Related work

2.1 Introduction

The NoSQL database paradigm has shifted how we develop modern web and mobile applications, and provides a versatile alternative to the rigidity of traditional relational database systems. Given today's needs for scalable services, the use of datastores such as Firestore, and Realtime Database becomes increasingly important. Such datastores have undoubtedly influenced the software architecture and development practices used in modern software development. This section focuses on a discussion of various types of NoSQL databases and their characteristics, as well as related works relevant to the domain of this study through a review of relevant academic literature.

2.2 NoSQL Datastores

NoSQL datastores can be classified into four different categories (Sadalage and Fowler, 2012):

1. Key-value datastores
2. Document datastores
3. Column/Family datastores

4. Graph datastores

2.2.1 Key-Value datastores

Key-Value datastores such as Redis, Amazon DynamoDB, and the Realtime database represent a simple form of NoSQL. In this approach, data is stored by forming pairs of keys and values, which can achieve very high performance and scalability due to the small order of growth of map-like data structures. This allows very fast read and write operations, albeit with limited support for complex queries (DeCandia et al., 2007). The simplicity and speed of such datastores makes them suitable for use in real-time pipelines such as those found in Massively Multiplayer Online Game backends (MMOGs) (Kasenides and Paspallis, 2022; Kasenides, 2023), as well as for data caching and session storage. Despite that, the limited schema enforcement and query capabilities means that applications must handle any additional needs for querying data manually, as well as verify integrity constraints and handling serialization/deserialization logic, thereby leading to development and performance overheads, especially in large-scale systems (Pokorny, 2013).

2.2.2 Document datastores

Document-based datastores such as MongoDB and Firestore store data as documents using a JSON-like structure. This allows developers to create more complex structures by nesting documents within each other and organizing documents into collections, which enables them to model a wider variety of information. Furthermore, document-based datastores can offer support for more complex queries compared to key-value datastores, but are also limited compared to query support in RDBMSs. As with key-value datastores, document-based datastores do not apply any rigid schemas to the data (Pokorny, 2013), which in turn addresses some of the limitations of relational databases in rapidly evolving software. However, this can also turn into a disadvantage, as the lack of schema enforcement introduces various challenges in terms of consistency and maintainability – especially for applications that grow

quickly in complexity over time (Stonebraker, 2010).

2.2.3 Column-family datastores

Column-family stores such as Apache Cassandra and HBase attempt to merge some principles from NoSQL and RDBMSs by extending the key-value approach to organizing data into column families and rows. This approach is particularly useful for applications which induce write-heavy workloads and time-sorted data, as it can efficiently distribute partitions across computing clusters while maintaining a relatively high level of availability (Lakshman and Malik, 2010). For instance, Cassandra uses *tunable consistency* and supports *denormalized schemas* to optimize queries, which makes it suitable for use in scalable services. Despite this, learning how to efficiently model data in this approach can require a deep understanding of advanced scalability concepts such as *partitioning*, *clustering*, and *consistency tradeoffs* – thereby impacting the ability of software developers to quickly set up and utilize this approach, as well as evolve its schema over time (Han et al., 2011).

2.2.4 Graph datastores

Graph datastores like Neo4J and Amazon Neptune can enable the storage of data that is strongly interconnected. These consist of *nodes* and *edges* with their associated data, which enables the execution of complex graph-based queries (Rodriguez, 2015). This type of datastore is useful in domains such as social networking, fraud detection, and knowledge graphs, as they can outperform the document-based and relational databases both in terms of performance and query flexibility. Despite this, graph-based datastores have limited scalability and their specialized query systems and languages can introduce a steep learning curve for software developers (Pokorny, 2013).

2.2.5 Other characteristics

A notable feature of various types of NoSQL datastores is support for *eventual consistency*. In contrast with the strong consistency employed by relational systems that comply with the ACID (Atomicity, Consistency, Isolation, Durability) principles, eventually consistent databases can provide advantages in performance, scalability, and data availability. Such systems focus on meeting the availability and partition tolerance requirements for two of the properties of the CAP theorem (Consistency, Availability, Partition Tolerance) in sacrifice of strong consistency (Lee et al., 2021). Another characteristic of NoSQL datastores is that in many cases there is a need to perform client-side queries/joins as there is poor support for database-side queries. This can be particularly expensive both in terms of cost but also in terms of resource utilization, especially when large amounts of data must be extracted. Finally, performance enhancing techniques such as normalization become more obscure, as these are not as well-defined as in their RDBMS counterparts (Moniruzzaman and Hossain, 2013), which can impact code complexity and maintainability.

2.3 Firebase

Firebase, initially developed by Firebase Inc. and later acquired by Google, is perhaps one of the most known Backend-as-a-Service (BaaS) platforms that support software development. Firebase offers two cloud-hosted NoSQL databases called Firestore and Realtime database. These databases are popular options for developers when it comes to realizing application data persistence on the cloud. They offer document/key-value storage, features such as real-time data synchronization, offline capabilities, and integration with security policies and authentication services. This high level of integration with other services, makes these databases suitable for rapid prototyping, and especially attractive for cross-platform application development (Moroney, 2017).

Firebase’s networking architecture is significantly different from the traditional client-

server model. Rather than employing a unidirectional data flow, Firebase utilizes RESTful APIs over HTTP to open persistent WebSocket connections that enable real-time bidirectional traffic to and from client devices. This approach significantly improves performance by decreasing latency but also simplifies application logic — especially for collaborative or real-time applications where data must be synchronized quickly. While these are major advantages, Firebase introduces some considerations regarding data security, caching policies, and conflict resolution (Moroney, 2017).

Some studies have explored the impact of Firebase on scalability and performance. For example, Gajendran and Ghosh (2017) evaluated the Realtime database using different workloads and measured its performance bottlenecks when retrieving nested data and while experiencing write surges in within large datasets. In previous work, Firebase has been evaluated in the context of real-time distributed systems such as MMOG backends, exploring how such databases can cope with the performance and scalability requirements in update-heavy environments (Kasenides and Paspallis, 2022). Furthermore, Pavlov et al. (2016) also explored the cost implications of using Firebase databases and the vendor lock-in risks associated with Firebase and similar platforms.

2.4 ODM tools

The gap between object-oriented programming design and database structures is known as *impedance mismatch* (Zhang et al., 2025). To bridge this gap, developers employ various Object-Document Mapping (ODM) frameworks which automate the serialization and deserialization to and from in-memory objects to persistent data in the database (e.g., documents). Examples of such ODM tools are Mongoose for MongoDB and Morphia, but also some community-developed Firebase ODM tools such as `firestore_odm` and `Typesaurus`.

ODM tools can provide a variety of features. For instance, Mongoose provides schema validation mechanisms and query-building APIs that help to abstract the underlying data

structure. Such abstractions can improve code maintainability and reduce the code needed to manage data (Barker, 2013). When using Firebase, developers rely on libraries which integrate with the Firestore or Realtime Database SDK, but such tools frequently lack standardized schema enforcement. Typesaurus, a JavaScript library, attempts to bridge this gap by offering schema and type safety for Firestore interactions, while the `firestore_odm` package for Flutter/Dart attempts to offer serialization/deserialization support, but lacks support for object-oriented transactions and inheritance in its data models (Moroney, 2017). Some ODM tools also provide more advanced features such as tracking changes, concurrency controls, and lifecycle event hooks. Such capabilities may align with developer expectations from ORM tools used in relational database, but can also conflict with the eventual consistency and schema flexibility approach used in NoSQL databases (Pokorny, 2013).

2.5 Challenges

ODM tools can face challenges in terms of code maintainability, scalability, and performance. Various works have identified such challenges when developing applications using NoSQL databases and ODM frameworks.

Schema-less designs employed in NoSQL datastores tend to increase the likelihood of data inconsistencies and misrepresentations, and typically require additional validation on the client-side (Moniruzzaman and Hossain, 2013). From a maintainability standpoint, this means that application code can become increasingly harder to evolve over time as developers start making implicit assumptions about the data and these spread across the code Pokorny (2013).

The primary motivation for adopting a NoSQL datastore is scalability. However, realizing horizontal scalability does not just depend on which datastore is used, but also on careful data modelling, using partitioning strategies, and understanding the trade-offs between availability and consistency (Han et al., 2011; Kasenides and Paspallis, 2022). For

example, Firebase inherently provides horizontal scaling for data, but also recommends to developers to avoid excessively nested structures and query hotspots which can degrade performance under scale (Moroney, 2017). Therefore, ODM tools must be configured in a way such that they avoid these problems and can enable applications to take full advantage of NoSQL datastore horizontal scaling without detriment to scalability.

The performance of a system is another key consideration that can complicate design choices. Abstractions introduced by ODM tools can introduce overheads that impact processing time and latency, especially during serialization and deserialization and query translation. This is particularly important when handling large volumes of data or running complex queries (Barker, 2013), but also while synchronizing under heavy concurrent access (Gajendran and Ghosh, 2017; Kasenides, 2023).

2.6 Impact on software development

The adoption of Agile methodologies has favored the use of rapid prototyping and continuous deployment of systems. Developers have utilized the flexible schemas of NoSQL datastores and the features of BaaS platforms, which align well with these objectives (Sadallage and Fowler, 2012). Consequently, the real-time capabilities of Firebase’s datastores have accelerated the development of many collaborative and real-time applications in various domains such as education, gaming, and research (Moroney, 2017; Kasenides and Paspallis, 2022; Kasenides et al., 2023; Schroeder et al., 2025).

However, the cost of this agility can materialize as increased technical debt, if development teams fail to establish standards for robustly validating data. The lack of mature ODM frameworks more generally, but more specifically for the Dart/Flutter development environment and Firebase forces developers to implement custom logic for mapping objects to documents and vice versa, for each individual class in each individual app. This significantly hampers maintainability and increases the likelihood of introducing bugs that may

go undetected for large periods of time (Pokorny, 2013).

2.7 Summary

Existing research has shown that NoSQL datastores like Firestore and Realtime database can enable the development of highly-scalable and available services. The use of ODM tools has the potential to reshape the software architecture of applications by prioritizing developer productivity and maintainability, while taking advantage of NoSQL's scalability and flexibility. Despite that, it is evident that ODM tools can introduce new challenges in terms of data consistency, maintainability, and performance, which must be addressed for them to be considered reliable and productive tools.

Chapter 3

Planning and Methodology

3.1 Introduction

This chapter discusses the approach taken to develop Firestorm. It outlines the project's planning and stages, specifies the software requirements, and presents the research methodology.

3.2 Research design

The *design science* paradigm for software development is typically employed to design and implement information systems where the primary product is an artifact. This paradigm emphasizes the iterative construction of an artifact, and its evaluation and refinement through several stages and through the application of various technological solutions (Vom Brocke et al., 2020). This particular paradigm is considered suitable for this project, because the objective is not to observe or analyze an existing phenomenon, but to create and validate a novel solution to an established software engineering problem (Hevner et al., 2004). The design science paradigm is applied to this project to divide it into stages:

1. **Problem Identification and Motivation:** There is a lack of mature ODM tools for Flutter applications that employ Firebase as a backend and utilize Firestore and/or the Realtime database.
2. **Objective Definition:** The objectives of Firestorm are ease of use, minimal overheads, improved code maintainability, compatibility with both Firestore and Realtime Database, and clear documentation.
3. **Design and Development:** Firestorm will be designed and implemented iteratively and incrementally, with basic features implemented first, and more complex features added later. Support for Firestore will be implemented first, followed by the Realtime database.
4. **Demonstration:** A demonstrator, testing, and evaluation app will host code showcasing the use of the library within a Flutter app, along with code for testing and evaluating its operations.
5. **Testing & Evaluation:** Unit and integration testing will validate the behavior of Firestorm's operations, whereas the evaluation will measure its effectiveness in terms of code maintainability and performance.
6. **Communication:** The findings will be reported in this thesis report.

The project stages are summarized in Table 3.1.

Stage	Expected duration	Start date
Requirements specification	7 days	26/05/2025
Design	10 days (intermittent)	02/06/2025
Implementation	20 days	09/06/2025
Testing	5 days (intermittent)	16/06/2025
Evaluation	3 days	01/07/2025
Report & Documentation	10 days (intermittent)	16/06/2025

Table 3.1: A summary of the timeline and stages in the project.

3.3 Development Methodology & Deliverables

3.3.1 Development methodology

A suitable development approach for this project is a hybrid between SDLC (Software Development Lifecycle) and Agile methodology that is adjusted specifically for use by a single software engineer. Based on specific parts of the project, either the SDLC or the Agile methodology can be used. For instance, the first stages of the project such as the requirements specification and initial design may benefit from the more solid structure offered by the SDLC methodology, which enables planning, requirements gathering, and some parts of design to be done in solid steps. This can ensure that the idea matures during these stages before the implementation lifecycle begins. On the other hand, the Agile methodology provides greater flexibility to adjust to changing requirements and adapt to issues that arise during development. This flexibility can lead to an improved focus on important tasks, therefore leading to a higher quality product (Kautz et al., 2014). During the main stages of the project, where the artifact is designed, developed, and tested, the project can benefit from an Agile approach by utilizing sprints, along with incremental and iterative development (Riesener et al., 2021). This can be enhanced with task prioritization using the MoSCoW technique (Kravchenko et al., 2022) and timeboxing, to ensure that priority requirements are implemented in the initial sprints, and that the appropriate amount of time is spent working on each requirement.

3.3.2 Deliverables & Timeline

The project's deliverables consist of:

1. A Dart package (Firestorm) that can be installed in any Dart/Flutter application, enabling the use of ODM concepts and a generic data access API to fulfill various data management requirements (Pitoura and Samaras, 2012).

as well as an analysis of existing tools and their limitations mentioned in Chapter 2. Each of these requirements is prioritized and sorted in Tables 3.2 and 3.3 using the MoSCoW technique (Kravchenko et al., 2022).

3.4.1 Functional requirements

Table 3.2: Functional requirements, prioritized using MoSCoW.

ID	Requirement	Priority
F1	Ability to define data models as annotated Dart classes	M
F2	Static type checking of data classes, for both datastores	M
F3	Verify conformance of data classes to Firestore data model	M
F4	Offer a way to register classes for use with Firestore	M
F5	Generate serialization/deserialization methods for data classes	M
F6	Utilize serialization/deserialization methods internally, without explicit developer use	M
F7	Support both Firestore and Realtime database APIs with seamless use/transition between the two	M
F8	Provide an object-oriented API that allows developers to interact with objects rather than raw data	M
F9	Enable the use of inheritance in data classes	M
F10	Enable the use of data classes within other data classes (Composition)	M
F11	Offer basic CRUD operations for Firestore	M
F12	Offer basic CRUD operations for Realtime database	M
F13	Provide ways to handle errors during operations	M
F14	Provide an asynchronous API which can be used in blocking or non-blocking mode	M
F15	Provide an API for queries and filtering for Firestore	M
F16	Provide an API for queries and filtering for Realtime database	M
F17	Offer batch operations (multi-read, write, update, delete) for Firestore	M
F18	Offer batch operations (multi-read, write, update, delete) for Realtime database	M
F19	Provide support for transactions in Firestore	M

Continued on next page

ID	Requirement	Priority
F20	Allow the use of enums in data classes	M
F21	Allow the quick creation of randomly generated IDs for database objects	M
F22	Provide support for object-oriented real-time listener functions in Firestore	S
F23	Provide support for object-oriented real-time listener functions in Realtime database	S
F24	Enable developers to retrieve raw references to documents and work with the traditional Firestore API	S
F25	Enable developers to retrieve raw references to keys and work with the traditional Realtime database API	S
F26	Provide an easy way to enable Firestore caching through Firestore	S
F27	Provide an easy way to enable Realtime database caching through Firestore	S
F28	Provide an easy way to configure Firestore in online/offline mode selectively through Firestore	S
F29	Provide an easy way to shut down the Firestore connection and deallocate resources through Firestore	S
F30	Provide an easy way to shut down the Realtime database connection and deallocate resources through Firestore	S
F31	Provide support for creating subcollections in Firestore	C
F32	Provide support for creating subcollections in Realtime database	C
F33	Implement a pagination API for Firestore	C
F34	Implement a pagination API for Realtime database	C
F35	Utilize multithreading or concurrent operations where appropriate to improve performance	C
F36	Provide an easy way to use the Firestore emulator through Firestore	W
F37	Provide an easy way to use the Realtime database emulator through Firestore	W
F38	Provide a way to customize the name of collection for each class in Firestore and Realtime database	W
F39	Support for Firestore sentinels	W

3.4.2 Non-functional Requirements

Table 3.3: Non-functional requirements, prioritized using MoSCoW.

ID	Requirement	Priority
NF1	Reduce boilerplate code (e.g., for serialization/deserialization)	M
NF2	Provide a better level of code maintainability compared to the traditional Firebase APIs	M
NF3	Achieve low performance overheads during database operations	M
NF4	Ensure that data mapping does not corrupt the database	M
NF5	Provide comprehensive documentation to learn the data access API and how to generate models	M
NF6	Must work on the Web	M
NF7	Must work across multiple mobile platforms including Android and iOS	M
NF8	Maintain compatibility with the security rules of Firestore and Realtime database	S
NF9	Maintain the graceful degradation of services of traditional Firebase APIs	S
NF10	Provide an extensible API that can be extended in the future	S
NF11	Must work on Desktop platforms including Windows and MacOS	S
NF12	Follow Dart and Flutter conventions and terminology to maximize adoptability	C
NF13	Support the latest stable Firebase SDKs and be interoperable with their APIs	C

The above requirements will be implemented in three different sprints:

1. **Sprint 1 (09/06/2025 - 15/06/2025):**

Minimum Viable Product with core features such as defining data models, type check-

ing, code generators, and basic CRUD operations (F1-F14)

2. Sprint 2 (16/06/2025 - 22/06/2025):

Intermediate sprint with more advanced features such as support for queries and transactions, batch updates, etc. (F15-F23)

3. Sprint 3 (23/06/2025 - 01/07/2025):

Full Beta prototype with additional features such as object-based real-time listeners, configuration options, and more. (F24-F39)

3.5 Tools & Technologies

This project relies on the use of certain tools and technologies which are required to undertake this project. Firstly, Dart and Flutter SDKs are used as part of the development environment, as the project will be created as a Flutter package. Secondly, the `firebase_core`, `firebase_database`, and `cloud_firestore` packages for the Realtime database and Firestore are needed, as they serve as the foundation of the package onto which the ODM layer will be constructed. For code generation the `build_runner`, `source_gen`, `analyzer`, and `build` packages are very important, as they provide the necessary functionality to scan and analyze code, and to generate various functions related to serialization/deserialization at compile time. Git and GitHub can be used as a version control system which enables the incremental development of Firestorm. Along with Git, GitHub's issues, projects, and pages features can serve as a way to manage project, track issues and bugs, and to host documentation. The development environment consists of Android Studio, which provides support for Flutter using a relevant plugin, the Android SDK and Emulator which serve as a testing platform, as well as the Java SDK, which is needed to run Android-based applications. For testing purposes, Flutter's `flutter_test` and `integration_test` packages along with Firebase's emulator suite can be used to conduct testing by running a Flutter app as part of an Android ecosystem, which interacts with local Firestore and Realtime database emulators.

Once testing is finished, the `pub` command line tool and `pub.dev`, which is a Dart and Flutter package repository will be used to deploy the package as a public beta prototype.

The tools, technologies, and techniques utilized for evaluation purposes are discussed in Section 3.7.

3.6 Risk management

The entire project is undertaken in the very short timeframe of approximately 8 weeks, and therefore any risk associated with the project must be planned for and mitigated carefully for the project to be successful. Table 3.4 summarizes the main risks and mitigation strategies that could be used to address them.

Risk	Impact(1–5)	Probability (1–5)	Mitigation Strategy
Underestimating time for complex tasks such as static checking, code generation, implementing support for OOP concepts such as inheritance etc.	5	4	Prioritize core functionality first, Use an extreme working hour schedule to ensure that tasks are finished ahead of time with time to spare for unforeseen circumstances
Serialization overhead exceeds acceptable thresholds	4	3	Test and profile the package early, optimize critical code paths, utilize concurrency.
Inadequate testing fails to catch bugs	2	5	Robust test coverage with integration tests, release as an experimental beta first
Incompatibility with Firebase SDKs & updates	5	1	Maintain a modular architecture and develop a expandable API, utilize up-to-date SDKs
Low adoption due to complexity and inadequate documentation	1	3	Provide a clear documentation and API reference, supplemented by examples and a demonstrator app

Table 3.4: Identified risks with impact, probability, and mitigation strategies.

3.7 Evaluation strategy

The evaluation of Firestorm entails three different components to ensure a more general assessment of its effectiveness and suitability for development. The following sections describe each of these components and discuss the strategies used to obtain measurements and answer research questions. Further details about the exact procedures are discussed in Chapter 7.

3.7.1 Performance

The performance evaluation focuses on measuring the overheads introduced by Firestorm’s object-oriented API, compared to the native non-object-oriented Firestore and Realtime database API operations. A benchmarking process aims to record the *processing time* of matching types of operations executed both using the Firestorm and Native APIs. This can be achieved by running code within the demonstrator app, which can run within an Android Emulator. The app can be connected to a running Firebase Emulator suite, that will be hosting only one of the two datastores at a time to avoid any performance degradation. The host device will feature an Intel i7 11700K processor with 32GB of RAM and an NVMe SSD with read and write speeds exceeding 5000MB/s. The results obtained from this experiment will be recorded in milliseconds, will be categorized for each operation, tabulated, and analyzed to identify any performance degradation or improvement compared to the native APIs. A *performance degradation threshold* is set to -10% of the native API performance. Performance that is worse than this threshold will be regarded as unsatisfactory for an operation, which means that the implementation of that operation must be reviewed at a later stage. This form of relative measurements ensures that the impact of any secondary factors is negated. Such factors include the performance characteristics of the host machine, the capacity and performance of the Android and Firebase emulators, network speed and throughput, processing spikes on the host machine, performance degradation due to Flutter’s debugging mechanisms, and more, in line with recommendations from previous studies

(Reniers et al., 2017). Other experimental features are also considered, such as the use of a common data model across all operations to ensure a consistent data size, the creation of nearly identical code with the same structure and clauses, and the use of multiple trials as part of the same experiment to increase the reliability of the data.

3.7.2 Maintainability

The second component of the evaluation is an assessment of the maintainability of the code when using Firestore operations. To achieve this the demonstrator app will be used as a platform where groups of code snippets can be written. In each group of code snippets, one datastore operation will be tested, broken into two or three implementations — one for the Firestore API and one (or two where applicable) for the Native APIs for Firestore and Realtime database. As in the performance component, the code will utilize a single data entity to ensure consistency, and the code written will utilize identical code constructs. Various CRUD operations, batch operations, but also more advanced features such as batch writes, transactions, and real-time listeners will be tested. The maintainability of each code snippet will be measured using the Maintainability Index (MI) metric (Heričko and Šumak, 2023), which composes of (a) the Lines of Source Code (SLOC), (b) Halstead Volume (HV), and (c) Cyclomatic complexity (CC). The formula used to determine the Maintainability Index of a snippet of code is shown in Equation 3.7.2.

$$\begin{aligned}
\text{Maintainability Index} &= 171 \\
&- 5.2 \times \ln(\text{Halstead Volume}) \\
&- 0.23 \times \text{Cyclomatic Complexity} \\
&- 16.2 \times \ln(\text{Lines of Code})
\end{aligned} \tag{3.1}$$

Figure 3.2: Maintainability Index formula.

To measure Halstead Volume, Cyclomatic Complexity, and Lines of Code, the experiment will involve the use of an AI agent which conducts code analysis. The agent will receive code snippets as text-based input, calculate code metrics such as HV, CC, and SLOC while also reporting the process followed (i.e., tokens/paths considered, etc.). The output will report the code metrics discussed above, thereby enabling their tabulation and analysis. Based on the data gathered, a quantitative analysis can lead to conclusions on whether using Firestorm improves or worsens code maintainability compared to the native APIs, and whether it has the potential to simplify the code maintenance process.

3.7.3 Feature coverage comparative analysis

The third component of the evaluation is a feature coverage comparative analysis. Various similar tools will be considered in multiple programming environments such as Dart, Java, NodeJS, and more. The analysis also identify popular ORM/ODM features, and will consider basic feature coverage, as well as more advanced features. The purpose of this analysis is twofold. Firstly, it aims to explore the features of other ODM tools already in use and enable a comparison between them and against Firestorm. Secondly, it aims to determine Firestorm’s percentage of feature coverage, situate it within the broader ecosystem of ORM/ODM tools, and reveal its potential limitations.

3.8 Summary

This section outlines the plan for activities to be undertaken as part of this thesis. The development methodology, deliverables, and timeline are defined, and the requirements of Firestorm are specified and prioritized. The section further discusses tools, techniques, and technologies used in the implementation and evaluation of the project, and establishes the risk management and evaluation strategy. This sets the precedent for the next section, which presents the design elements of Firestorm and discusses key architectural principles.

Chapter 4

Design

4.1 Introduction

This chapter discusses the design elements of Firestorm. It delves into the decisions taken to divide the package into layers and components using modular design principles, with all of these layers forming a unified software architecture (Hare and Kaplan, 2017). The design of the data access API is discussed, along with the rules established regarding data definition and organization. The design section also presents code design decisions based on various elements of the Dart programming language. The overall design objective of this project is to provide an extensible, modular data access API that can offer high learnability and code maintainability in par with the requirements outlined in Chapter 3.

4.2 Overview

Figure 4.1 demonstrates the intended setup and usage workflow for Firestorm to be used in a Flutter project. The diagram distinguishes this process into two phases: setup and usage. The setup process (blue) is a one-off step in which developers are intended to configure their project to use Firebase, whereas the usage process (green) is an iterative process where developers are expected to re-run several steps when applying data model updates to their

applications.

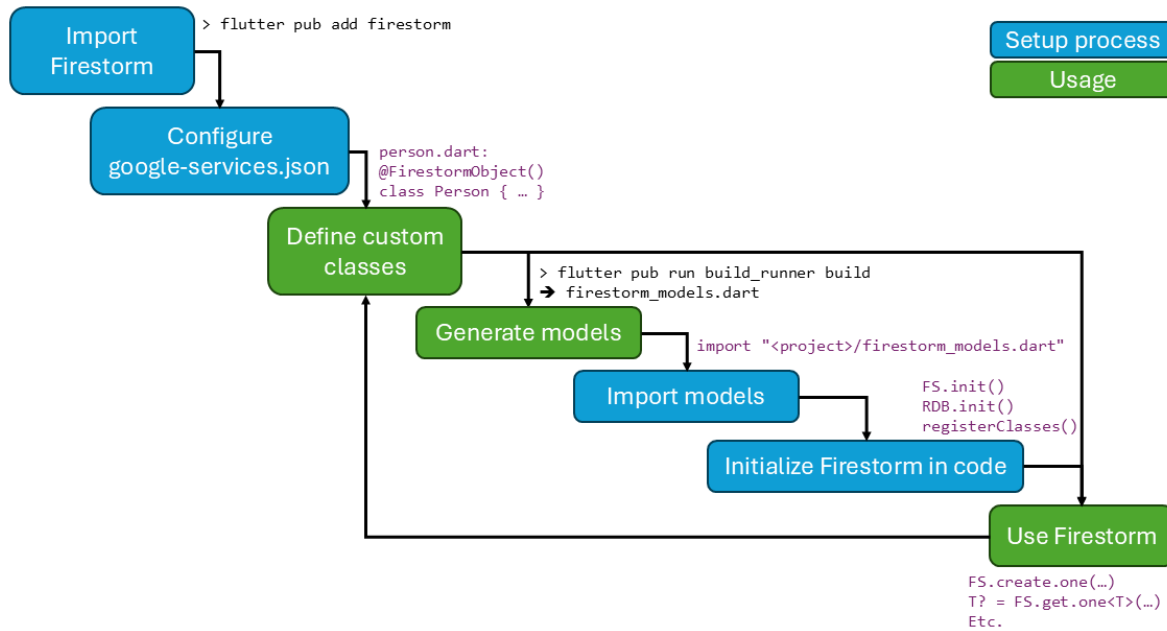


Figure 4.1: Firestore setup and usage workflow

The workflow begins by creating a Flutter project, where Firestore is imported as a dependency through the Dart package manager (pub). It is envisioned that this can be achieved by running the command `flutter pub add firestore` on the project terminal.

The second step involves the generation of a Firebase configuration file (`google-services.json`), which must be added to all Flutter sub-projects (i.e., Android, iOS, etc.) to create a connection to Firebase's services and access Firestore and Realtime Database. Following the correct configuration of this file, developers must then define their data entities in step 3. These are expected to be annotated Dart classes, which will allow code analysis tools to detect and analyze these entities. Considering that an initial data model has been defined, the developer must run the build process to analyze the entity classes and generate various pieces of code for serialization, deserialization, class identification, and more. The result of this process will be a single, generated file which contains adapter code and needs to be imported in step 5 by the developer to access critical functions needed to operate Firestore. Ideally, this file will only need to be imported once

(i.e., in the file containing the `main()` function) to register classes. After importing the generated code, the developer will need to initialize Firestore within their code (step 6). Since developers may opt to carry data operations using just one of the two datastores or both, the Firestore API must allow the initialization of a connection to either or both datastores in each case. Another important part of this step is class registration. Data entities will need to be registered with Firestore so they can be used in operations. Ideally, calling a single function should suffice to register all classes together, as they would have already been detected by code analysis tools. Finally, in step 7, developers can utilize Firestore to carry object-oriented data operations, such as creating and reading objects to and from the database.

As mentioned above, Steps 3, 4, and 7 are iterative steps. When a change is made to the data model, the generated code will become outdated and needs to be regenerated. Therefore, when there is a new definition in the data classes (step 3), step 4 must also be executed again to ensure that the application is using the latest descriptions from those classes. While Firestore’s design may facilitate such changes by allowing the code to still compile without regeneration even when there is a slight change to any of the data entities, significant changes may cause applications to crash when an inconsistent model is used.

4.3 Software Architecture components

The software architecture focuses on a layered approach that aims to separate concerns across multiple high-level layers within the package. This can improve the modularity of the package, facilitate the continuous integration of new features, and make testing easier. The package is divided into the following high-level software components:

- **Data modeling component**, which defines various utilities for data modeling such as annotations, type systems, and static type checkers.
- **Code generation component**, which includes tools used to generate code for class

detection and registration, and data serialization/deserialization.

- **Data access component**, which contains abstractions for various types of data operations that can be included within the data access API.
- **Firebase integration component**, which enables applications to quickly integrate with Firestore and the Realtime database to execute the operations defined in the data access component.

This software architecture ensures that various software components such as code generators and runtime logic remain as decoupled as possible, which can improve the maintainability of the package itself and enable isolated unit/integration testing.

4.4 Data modeling

4.4.1 Data types

Since Firestorm needs to support both the Firestore and Realtime database, it needs to conform to the corresponding data types for each of these datastores in the Dart programming language. Types which are not supported by each of these datastores should not be accepted into the corresponding serialization and deserialization mechanisms, and therefore, the tools which generate code must take this into consideration. To meet the data type constraints of these two datastores, the static checking components must therefore differentiate the validation of data classes, and accept them for use by each datastore based on the accepted data types of each datastore. Table 4.1 identifies the usable Dart data types which are accepted by Firestore and Realtime database.

4.4.2 Data organization and constraints

To create a consistent data structure within each respective database, data entities should follow some organization principles and constraints.

Firestore	Realtime Database
String	String
int	int
double	double
bool	bool
null	null
DateTime	List<dynamic>
GeoPoint*	Map<String, dynamic>
DocumentReference*	
Uint8List	
List<dynamic>	
Map<String, dynamic>	

Table 4.1: Supported Dart data types in Firestore and Realtime database.

Firstly, user-defined classes should be annotated with a Firestore-specific annotation which will enable code analyzer tools to detect these classes and perform static checking and code generation. Furthermore, both datastores utilize String-based document and object IDs to identify items within the database. Therefore, each object defined for use with Firestore must have a String-based `id` attribute, which serves this purpose. Documents and objects with the same ID are considered the same item, and so each object must carry a unique ID. Firestore will handle the structuring of data internally by maximizing its potential for horizontal scalability. This will be achieved through data *flattening*, which splits objects into different paths using *denormalization*. This means that data can be retrieved more efficiently in separate calls when it is needed, which can significantly increase horizontal scalability. Considering an example, if a database needs to store data related to chatrooms and messages that are related to these chatrooms, it can use a nested structure where chatroom messages are nested within each chatroom object. However, this means that when a query is made to retrieve information about the chatroom, messages for that chatroom will be retrieved as well, potentially causing significantly larger data reads and slower response times. For the Realtime database, which bills users based on data traffic rather than number of operations, this can also incur significantly higher costs. Thus, Firestore follows

a flattened structure, where objects are split into multiple paths. For example, chatroom information and messages could be stored independently, so messages can only be retrieved when needed to avoid these problems. This strategy can also be advantageous in Firestore, even though Firestore bills users per number of operations rather than data traffic. This is because Firestore uses a collection and document/subcollection structure, where data can be stored either as documents within collections, or as documents within subcollections of other documents. This approach allows queries to retrieve only the document data, and optionally retrieve subcollection data when needed, thus enabling more efficient operations. Firestorm will follow the flattened data organization approach by mapping each class to a Firestore collection or root tree node in the Realtime database. Each object of that class will be stored under that collection, with the ability to create subcollections in both datastores to store objects in different groups when needed.

Another requirement is handling support for object-oriented concepts such as inheritance and composition, which may allow developers to freely create complex data hierarchies. To support inheritance, entity analysis must detect whether an analyzed class is a derived class, since data from its base class will also need to be utilized in the derived class's data model. Developers may also opt to use composition. In this case, a defined class could be constructed by combining other classes' objects as attributes within it. For this to be supported, the role of static analysis is important, as the encapsulated attributes must belong to classes that are also considered valid for the two databases. Finally, it is important to state that this approach enables other object-oriented principles, such as polymorphism, where developers may choose to retrieve an item of a derived class from the database, but represent it as a base class object within their code whenever needed.

4.5 Validation and generation

4.5.1 Entity validation

An important step before generating code is to ensure that data entities defined by developers follow the constraints defined above. This ensures that database operations will be successful when they are executed and will not yield any errors related to invalid data types. This stage entails the use of static analysis to analyze code in defined classes. The analyzing components will first detect classes which are annotated with `@FirestoreObject`, which signifies that this is class which can be used with Firestore. Classes which are not annotated will not be checked, and thus be ignored. If such classes are used in Firestore operations through the data access API, Firestore will yield an exception, warning the developer that this class is not valid. The next stage in the pipeline is to check whether all annotated classes have a string-based ID field, and whether they are composed only of valid data types through their class hierarchy. Figure 4.2 and Algorithm 1 illustrate the steps involved in this validation pipeline.

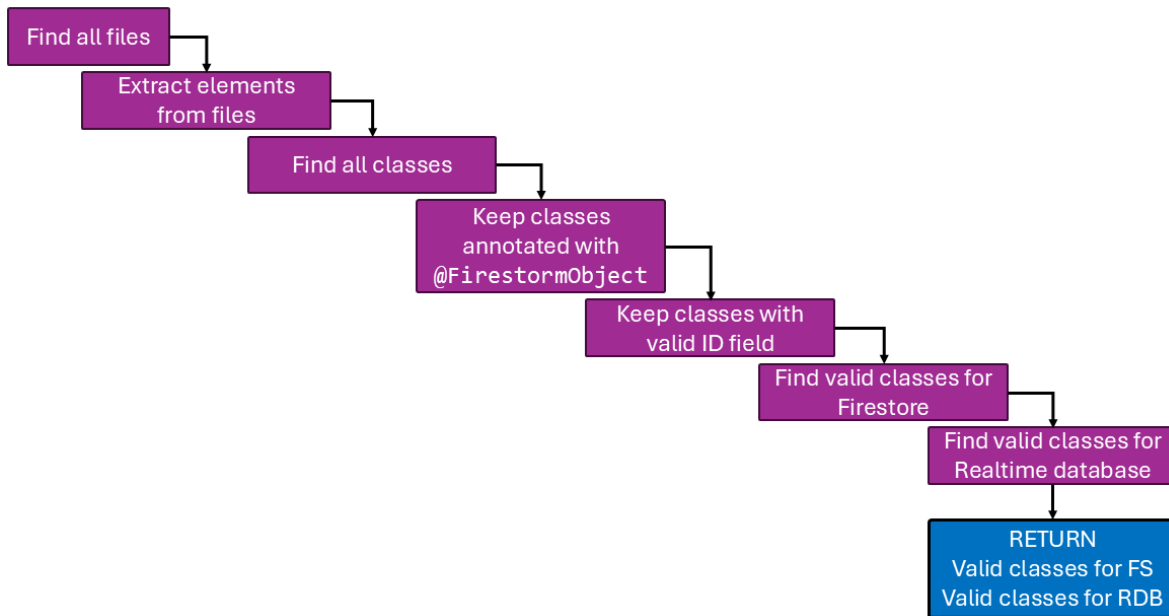


Figure 4.2: A visualization of the data entity validation pipeline.

Algorithm 1: Pseudocode showing the process of class validation.

```
(1) allFiles  $\leftarrow$  findAllFiles()
(2) allElements  $\leftarrow$  findAllElements(allFiles)
(3) classes  $\leftarrow$   $\emptyset$ 
(4) enums  $\leftarrow$   $\emptyset$ 
(5) foreach e  $\in$  allElements do
(6)   if e is Class then
(7)     | classes.add(e)
(8)   end
(9)   else if e is Enum then
(10)    | enums.add(e)
(11)  end
(12) end
(13) validClassesForFS  $\leftarrow$   $\emptyset$ 
(14) validClassesForRDB  $\leftarrow$   $\emptyset$ 
(15) foreach c in classes do
(16)   | validClasses  $\leftarrow$  findAnnotatedClasses(classes)
(17)   | validClasses  $\leftarrow$  findClassesWithIDField(validClasses)
(18)   | validClassesForFS  $\leftarrow$  checkFS(validClasses)
(19)   | validClassesForRDB  $\leftarrow$  checkRDB(validClasses)
(20) end
(21) return validClassesForFS, validClassesForRDB
```

4.5.2 Code generation

The code generation process involves four steps, which result to the creation of a single Dart file.

1. Header generation
2. Import generation
3. Extension generation
4. Registry function generation

During the header generation step, the generator will create a header comment at the beginning of the generated file which will warn developers that this code is generated and should not be modified. The header also contains other information such as the generation date and tool version.

While the code analyzer checks entities during entity validation, the asset ID of each entity can be stored. The asset ID can be used to identify an asset (i.e., a code element such as a class) within the package in which it was defined. This is important for code generation, as it can enable generated code to import elements from existing files — such as user-defined classes. During the *import generation* step, the generator will use these asset IDs to generate Dart `import` statements which will allow the code to use various elements that are needed for the code to function.

The extension generation step will facilitate the creation of Dart extensions, which add functionality to existing classes. In this case, this can be applied to the data entities defined by the user in order to extend their functionality with additional methods. The purpose of these methods will primarily be for serialization and deserialization. For example, methods such as `toMap()` and `fromMap()` could be generated within each class extension, allowing objects of each valid data entity to be serialized to a Map data structure or deserialized to

an object of that type. This step is important within the serialization mechanism, as both Firestore and the Realtime database receive and return data using Map data structures. In the final step of the generation process, the code generator will create data structures and functions which are needed to register classes. An important function in this case is the `registerClasses()` function, which will automatically register all valid classes with Firestore after it is initialized, as discussed in section 4.2. Other generated data structures may include function registries, which will contain generated entries mapping each data entity to its `toMap()` and `fromMap()` methods. These will be necessary, as such methods may not be visible statically through the data access API, which works with `dynamic` objects. As such, these structures can be used to determine the correct functions to call when an object has to be either serialized or deserialized at runtime. These concepts stem from the Registry design pattern, which enables the use of a global registry to manage and provide access to a shared pool of objects, services, or configurations (Mayvan et al., 2017). This pattern is coupled with the Strategy pattern to create pluggable components which are not discoverable until the code is compiled, thus enabling the use of the mentioned data structures and functions within application-specific code.

4.5.3 Data access API

The data access API provided by Firestore is perhaps one of the most critical components of the package. The API must use a clear, easily learnable, and maintainable design which can enable developers to utilize it effectively, and to maintain and improve the package in later stages. Several key principles are guiding the API's design:

- **Consistency:** The components included within the API must follow predictable patterns and naming conventions that make it easy for developers to discover key functionalities and remember them while developing their apps.
- **Extensibility:** The structure of the API should make it easy to extend its function-

ality with additional components and functions in the future.

- **Type safety:** While allowing for flexibility, the API must enforce either compile-time or run-time checks to ensure the correct use of data in operations and queries, according to the developer-defined data model.
- **Performance:** Any abstractions introduced by the API should introduce minimal overheads to reduce the package's impact on application performance. Performance-intensive operations should be optimized as much as possible.
- **Error prevention:** The API should make it hard for developers to perform queries which can result to high cost or damage to data, such as accidental deletes of all objects of the same type, or retrieving massive amounts of data at a single time.
- **Declarative approach:** A declarative design approach (Antoy and Hanus, 2005) should be used to enable developers to obtain data using queries, making it more familiar with concepts they are already familiar with, such as SQL queries.
- **Error specificity:** The API should produce clear errors which enable developers to take correct actions to debug their applications.
- **Testability:** The API should make it easy to continuously run and integrate new code verification tests, especially as the API grows.

Figure 4.3 shows an overview of the data access API. Several design decisions have been made while designing this API. Firstly, the API is divided into three main components. The `Firestore` class is designed to hold common functions used by both `Firestore` and `Realtime` database. Subsequently the `FS` and `RDB` classes serve as *facades* and are responsible for exposing code which handles operations for each of the two datastores. It is decided that these classes should not be derived from a parent class even though they have common operations, as this could limit the potential expansion of the package to support more datastores in the

future. The lack of Dart support for nested static classes (such as those supported by Java) necessitates the use of such facade classes to enable a *namespaced* design. Each of the two classes contains configuration and management methods, as well as the *data access delegates*. The data access delegates are classes which host functionality related to each type of datastore operation and can be accessed through properties defined within the `FS` and `RDB` classes respectively (e.g., `create`, `get`, etc.). For instance, the `create_delegate` object could enable developers to access its methods to create one or multiple objects within the database. This can result to a one-liner statement similar to `FS.create.one()`, with an object to be created being passed to the function to create a single document in the database. Another important reason for this type of API design is the fact that Dart does not support function overloading as it is a dynamically typed language. As a consequence, it is not possible to use the same function name to conduct the same type of operation (e.g., using `FS.create()` to pass both a single object and multiple objects). This is a significant difference from the Java-based Firestore API, which utilizes function overloading and varargs parameters to support such design. Hence, these language peculiarities necessitate the use of the Delegate pattern to ensure that multiple types of operations are accessible from a single place, are easily discoverable, and with concerns separated to individual classes rather than a single class.

4.6 Summary

This section discussed the software architecture and components of the Firestore package and presented an overview of the process of setting up and using Firestore. Important design considerations are also discussed, such as data modeling for the two datastores, Dart language peculiarities and constraints, the steps necessary for entity validation and code generation, and more. All these design considerations enable the implementation of Firestore, which is discussed in the next section.

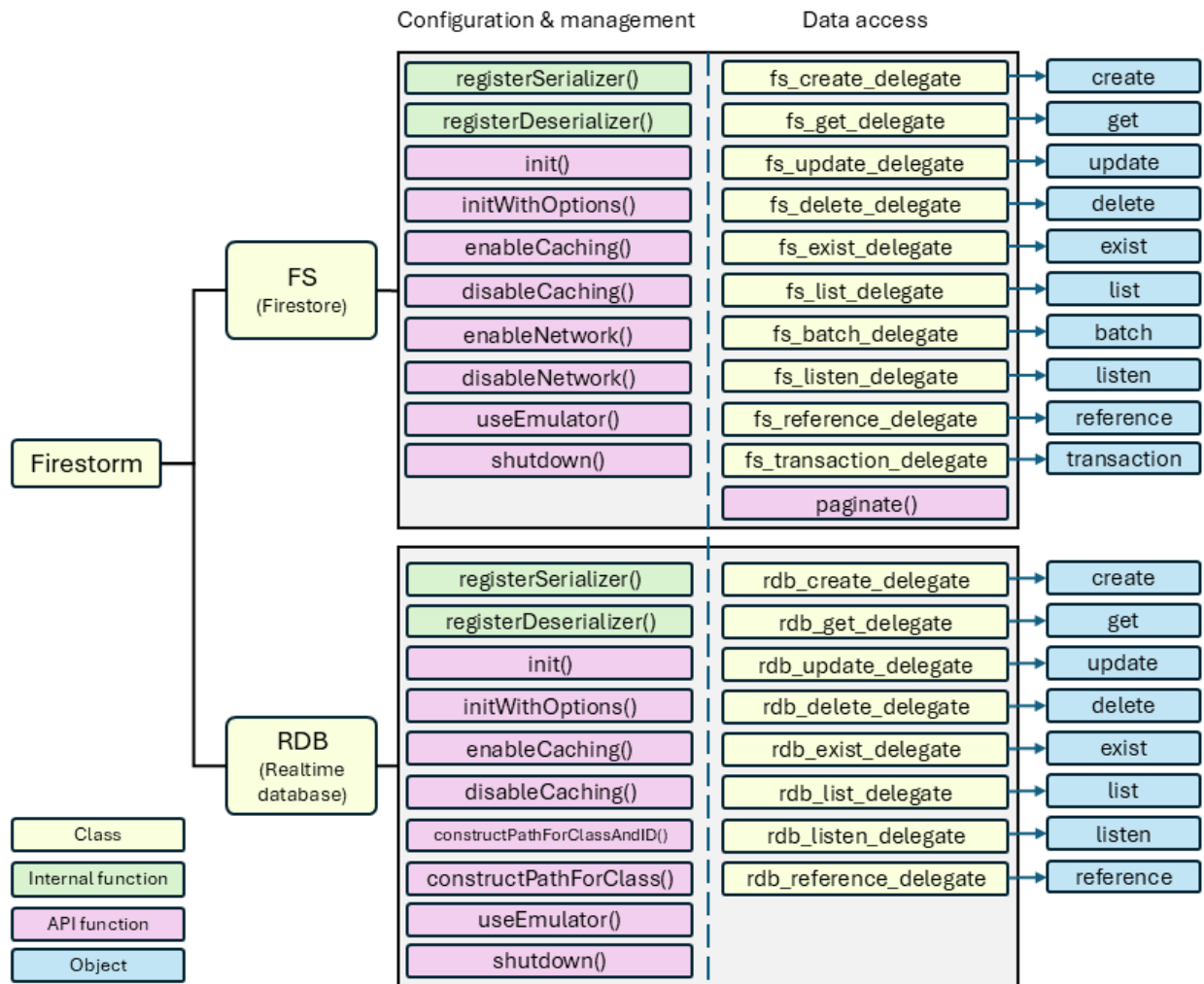


Figure 4.3: An overview of the data access API, including its classes, functions, and objects. More details about each delegate can be seen in Figure 4.4

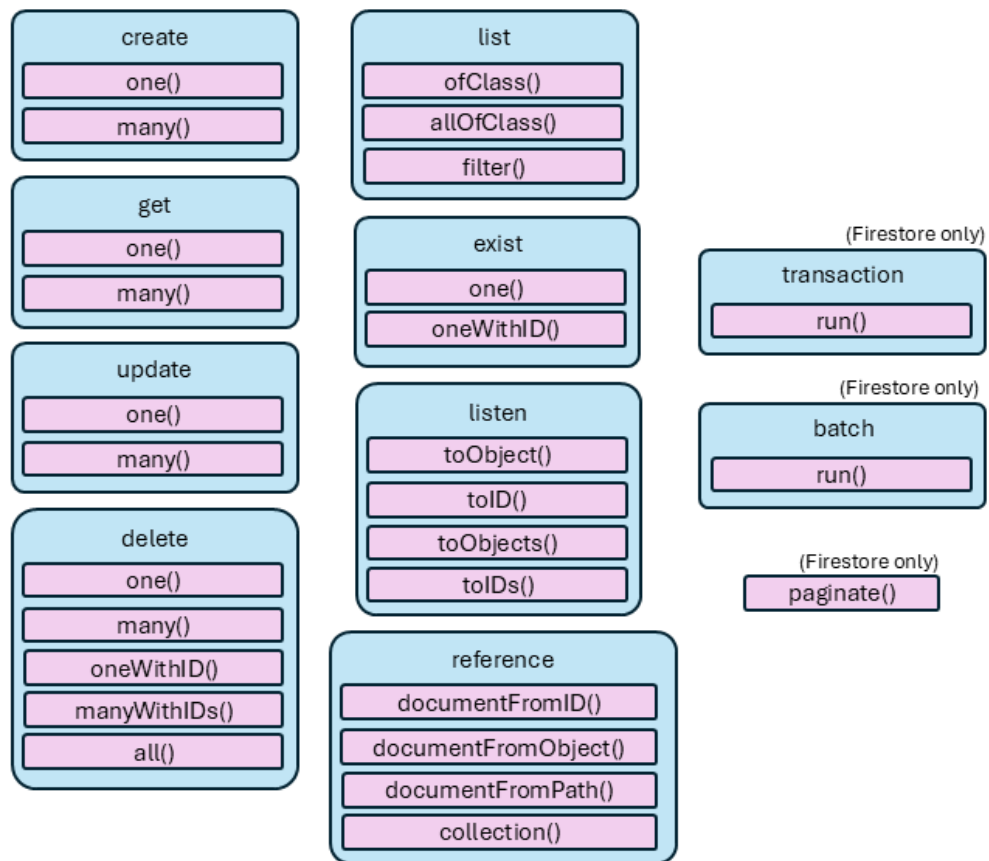


Figure 4.4: Functions within each delegate in the data access API.

Chapter 5

Implementation

The implementation phase uses the design considerations discussed in the previous chapter and transforms them into a functional Dart package meant for use in Flutter applications. This section discusses the structure and implementation of Firestore through various code listings and presents various challenges and their solutions. A demonstrator app and guide are also discussed, which demonstrate example uses of the package for guidance to developers.

5.1 Package structure

The project was created as a Flutter package and Android Studio served as the primary IDE (Integrated Development Environment). The code was tested using the Android Emulator and the Firebase Emulator Suite. The code in the project is divided into several Dart packages:

- `annotations`, which contains annotation definitions relevant to Firestore such as `@FirestoreObject`.
- `commons`, which contains abstractions for use by both Firestore and the Realtime database.
- `exceptions`, which defines exceptions relevant to Firestore errors such as

`InvalidClassException`.

- `fs`, which contains code relevant to Firestore operations.
- `gen`, which contains code relevant to data class validation and generation.
- `rdb`, which contains code relevant to Realtime database operations.
- `type`, containing utilities needed to define and check for valid data types in entity classes.
- `firestorm.dart`, which contains code for use in both datastores.

The modular structure of the package enables the use of various abstractions, which are especially useful when it comes to defining a common API for use across both datastores. This also supports the separation of concerns discussed in the previous chapter, increases code maintainability, and provides the potential to expand this package to other datastores in the future.

5.2 Model definitions & Validation

5.2.1 Defining data entities

The first step in enabling persistence in an application is the definition of a data entity by the developer. This describes the meaning of one or more data entities by defining its attributes and functionality. In this case, the functions of data entities are completely disregarded, and all focus is placed on the data attributes which make up an entity.

The definition of an entity class is done by creating a Dart class, like the one seen in Listing 1. To meet the data definition rules defined in section 4.4.2, each class must be annotated with `@FirestoreObject` (line 1), and contain an `id` attribute of type `String` (line 4).

```

1  @FirestoreObject()
2  class Person {
3
4      String id;
5      String firstname;
6      String lastname;
7      int age;
8      double height;
9      bool isEmployed;
10     List<String> friends;
11
12     Person(this.id, this.firstname, this.lastname, this.age, this.height,
13           this.isEmployed, this.friends);
14
15 }

```

Listing 1: An example of how an Firestore-managed entity can be defined.

5.2.2 Data entity validation

A major problem faced in the implementation of Firestore for Flutter is the lack of support for runtime reflection in Flutter applications. The lack of support for this capability is well-founded, as Reflection APIs are usually detrimental to application performance, and a common convention is to avoid reflection in client-oriented applications. However, this also means that it is impossible to analyze data types at runtime, something which was possible through the Java-based version of the package. Hence, the use of static analysis tools at compile time is used as an alternative, as in many popular Flutter packages that must analyze code such as `json_serializable`, `i18n`, and `reflectable`.

When data entities are defined, they must be validated before their use in datastore operations to avoid inconsistencies with the data types and structures supported by the respective datastores. This process is outlined in Algorithm 1, where the first step is the detection of all `.dart` files, and subsequently, the detection of all Dart elements, including classes and enumerators. The classes responsible for implementing this functionality are `FirestoreBuilder` and `ClassChecker`. The element detection process happens in the former, where Map data structures are used to store element information for later use, such as for import statements

and attribute analysis. Listing 2 shows a fragment of code from this file demonstrating how the code detects and categorizes code elements.

```
1 final Map<AssetId, Iterable<ClassElement>> allClasses = {};
2 final Map<ClassElement, AssetId> assetIDs = {};
3 final Map<EnumElement, AssetId> enumAssetIds = {};
4
5 //Read the classes from the files and store in allClasses map:
6 await for (final input in buildStep.findAssets(Glob('lib/**/*.dart'))) {
7   ↪ //iterates through all .dart files
8   if (!await buildStep.resolver.isLibrary(input)) {
9     continue;
10  }
11  final libraryElement = await buildStep.resolver.libraryFor(input);
12  final libraryReader = LibraryReader(libraryElement);
13  allClasses[input] = libraryReader.classes;
14
15  for (final aClass in libraryReader.classes) {
16    assetIDs[aClass] = input; //Map each class to its AssetId
17  }
18
19  //enums:
20  for (final aClass in libraryReader.enums) {
21    enumAssetIds[aClass] = input; //Map each enum to its AssetId
22  }
23 }
```

Listing 2: FirestormBuilder code fragment, used to detect Dart code elements and store them for later use.

Once Dart classes are detected, they undergo a set of filtering operations meant to remove classes which are not compatible for either datastore. In this case, a special holder class is used to distinguish between classes compatible with Firestore and those compatible with the Realtime database, as some classes can be valid for Firestore, but not valid for the Realtime database. Hence, the returning type called `ValidClassHolder` encapsulates two `Set` data structures called `fsValidClasses` and `rdbValidClasses`. A set of operations are executed from `ClassChecker`, which are responsible for filtering out the classes not compatible with either datastore, as seen in Listing 3.

```

1 ValidClassHolder allFilesClassHolder = ValidClassHolder.empty();
2 for (final pair in allClasses.entries) {
3     final Iterable<ClassElement> allClasses = pair.value;
4     var fileClassHolder = ClassChecker.filter(allClasses);
5     allFilesClassHolder.join(fileClassHolder);
6 }

```

Listing 3: Running filtering operations using `ClassChecker.filter()`.

In more detail, the `filter()` function calls `findAnnotatedClasses()` and `findClassesWithIDField()`, as well as the corresponding functions for each datastore `checkClassesForValidFSTypes()` and `checkClassesForValidRDBTypes()` to filter out invalid data entities, as seen in Listing 4.

```

1  ///Filters the provided iterable of ClassElements to find valid classes for
   ↪ Firestore.
2  ///Returns a ValidClassHolder containing two lists: one for Firestore valid
   ↪ classes and one for RDB valid classes.
3  static ValidClassHolder filter(final Iterable<ClassElement> classes) {
4      ///Find annotated classes:
5      List<ClassElement> validClasses = findAnnotatedClasses(classes);
6
7      ///Filter out classes that do not have an ID field of type String:
8      validClasses = findClassesWithIDField(validClasses);
9
10     ///Check classes for valid types:
11     Set<ClassElement> fsValidClasses =
   ↪ checkClassesForValidFSTypes(validClasses);
12     Set<ClassElement> rdbValidClasses =
   ↪ checkClassesForValidRDBTypes(validClasses);
13
14     ///Return a ValidClassHolder with the valid classes:
15     return ValidClassHolder(fsValidClasses, rdbValidClasses);
16 }

```

Listing 4: `ClassChecker.filter()`, showing the filtering process.

The use of subsequent functionality implemented in `FSTypes` and `RDBTypes` is important to dissect the data entities and check their validity. For instance, checking for a valid Firestore type entails the use of `FSTypes.isTypeSupported()`, which checks a Dart type for compatibility with the Firestore. There are several data types that are accepted, such as

those included in Table 4.1. However, the complexity of this task grows quickly as Firestore needs to also support additional cases:

- Support for enumerators
- List and Map attributes of which the typed arguments must conform to the valid data types defined.
- User-defined types defined as attributes within the class (composition).

This is by nature a recursive problem, as there could be an infinite amount of nested attributes for user-defined types in the class hierarchy. Listing 5 shows a code fragment within this function demonstrating how this is implemented.

5.3 Code generation

Once the code analysis is completed, two sets of valid data classes will be stored by a `ValidClassHolder` object, which are ready for use in code generation. The first step in this process is the generation of import statements, made possible by the use of the `AssetId` mappings created, as shown in Listing 2. The second step is the generation of Dart extensions for each valid data class. Listing 6 shows a code fragment within `FirestoreBuilder` responsible for carrying out these steps.

The `generateImports()` function works by generating one line of code for each file which needs to be imported for the generated code to function. This includes data entity classes defined by the developer, enumerators, and more. The `generateExtension()` function is responsible to generate a single extension at a time, one for each entity class defined. Within these extensions, there are two Boolean attributes, called `fsSupport` and `rdbSupport`, which indicate which datastores this entity is compatible with. These attributes can be used by the data access API or developers directly to determine whether this is a valid type for each datastore.

```

1  //...
2  else if (type.isDartCoreList) { //lists, must contain supported elements
3      final listType = type as InterfaceType;
4      if (listType.typeArguments.length == 1 &&
5          isTypeSupported(listType.typeArguments[0])) {
6          return true;
7      }
8  }
9  else if (type.isDartCoreMap) { //maps, must have String keys and supported
    ↪ values
10     final mapType = type as InterfaceType;
11     if (mapType.typeArguments.length == 2 &&
12         mapType.typeArguments[0].isDartCoreString &&
13         isTypeSupported(mapType.typeArguments[1])) {
14         return true;
15     }
16 }
17 //check user-defined types:
18 else if (type.element != null && !type.element!.library!.isDartCore && type is
    ↪ InterfaceType) {
19     final element = type.element;
20     if (element is ClassElement) {
21         List<FieldElement> allFields = [];
22         allFields.addAll(element.fields);
23         for (final parent in element.allSupertypes) { //Check parent classes too
24             if (parent.getDisplayString() != 'Object') { //Avoid the Object class
25                 allFields.addAll(parent.element.fields);
26             }
27         }
28         List<bool> fieldsSupported = [];
29         for (var field in allFields) {
30             if (!field.isStatic) {
31                 bool b = isTypeSupported(field.type);
32                 fieldsSupported.add(b);
33             }
34         }
35         //Return true only if all fields are supported
36         for (final bool b in fieldsSupported) {
37             if (!b) {
38                 return false;
39             }
40         }
41         return true;
42     }
43 }

```

Listing 5: `FSTypes.isTypeSupported()` code fragment demonstrating the use of recursion to analyze class hierarchies for valid attributes.

```

1 //For every valid class, generate necessary imports and its extension:
2 for (final validClass in allFilesClassHolder.getAllValidClasses()) {
3
4     //Add into the import buffer, if there are any classes in this file
5     ImportGenerator.generateImports(importsBuffer, assetIDs[validClass]!);
6
7     //Generate extensions:
8     ExtensionGenerator.generateExtension(
9         classBuffer,
10        validClass,
11        allFilesClassHolder.hasFSSupport(validClass),
12        allFilesClassHolder.hasRDBSupport(validClass)
13    );
14 }

```

Listing 6: FirestormBuilder fragment showing the generation of imports and extensions.

These extensions also contain serialization and deserialization methods. The `toMap()` method serializes entity class attributes into a Map data structure, which can be used by Firestore and Realtime database in data operations. This method is created by first inspecting class constructor parameters as well as class attributes, and then generating code which matches the values of these attributes onto Map entries, where they key is the name of each attribute. Developers can optionally select to exclude a field from being serialized into the map, and therefore avoid it from being written to the database, by using the `Exclude()` annotation near that field. Each metatype of attribute, is serialized in a different way, with the code handling nested user-defined types (composition) differently from enums and core Dart types. Listing 7 shows the generated `toMap()` method for the `Person` class defined in Listing 1.

Conversely, the `fromMap()` method is used to deserialize Map data structures which are received from the two datastores to entity class objects. This method is created by scanning each classe's parameters, and then instantiating and return an object of that type by specifying the values in the map as parameters in its constructor. However, this step is more complex, as Dart constructors can include both positional and named parameters, as well as optional parameters. For optional parameters, the values can be set to `null` whenever a

```

1 Map<String, dynamic> toMap() {
2   return {
3     'id': this.id,
4     'firstname': this.firstname,
5     'lastname': this.lastname,
6     'age': this.age,
7     'height': this.height,
8     'isEmployed': this.isEmployed,
9     'friends': this.friends,
10  };
11 }

```

Listing 7: Generated `toMap()` method for the `Person` class.

```

1 static Person fromMap(Map<String, dynamic> map) {
2   return Person(
3     map['id'] as String,
4     map['firstname'] as String,
5     map['lastname'] as String,
6     map['age'] as int,
7     map['height'] as double,
8     map['isEmployed'] as bool,
9     map['friends'] != null ? map['friends'].cast<String>() : [],
10  );
11 }

```

Listing 8: Generated `fromMap()` method for the `Person` class.

value is not present in the map. Positional and named parameters are handled differently, based on the syntax of the Dart programming language. Firstly, the positional parameters are handled by directly mapping them to their constructor positions. Then, code is generated for each named parameter by using its name and matched value. These processes must also consider excluded attributes, for which no code is generated. Listing 8 shows the generated `fromMap()` method for the `Person` class presented previously. It should be noted that this is a static method that can conveniently be used to instantiate an object without manually creating it first.

The complexity of handling data serialization differently depending on parameter types, the need to provide type-safe deserialization, and avoid runtime errors meant that the code

generation step was developed incrementally. For instance, the package initially supported only positional parameters, with support for named parameters being introduced in later sprints as a way to provide developers with the full flexibility of the Dart language when defining data entities. Among the various steps in the implementation of this project, the static analysis and code generation tasks were substantially more complex.

5.4 Delegate implementation

The developer-facing data access API is divided into delegate classes as shown in Figures 4.3 and 4.4. In the code, these are abstracted by the use of several Dart interfaces, as shown in Figure 5.1, which contain definitions for each function to be implemented by the derived datastore-specific delegates. For instance, both `FSCreateDelegate` and `RDBCCreateDelegate` implement the `CreateDelegate` interface. This abstraction offers the ability to consistently expand the data access API in the future to support more functions in each of the delegates, as well as to support more datastores under the same data access API.

Developers can use the data access API by using the `FS` and `RDB` classes respectively. The components within these classes are divided into the following groups:

1. **Serializer and Deserializer functions**, stored in Maps mapping each type to its `Serializer` and `Deserializer` functions. These are statically mapped at compile time, but need to be accessible during the runtime to provide serialization/deserialization functionality.
2. **Operation delegates**, which are defined as static attributes, enabling the use of various data operations in each datastore.
3. **Serializer and deserializer registration functions** which work to map each type to its serializer and deserializer and can be used at runtime.
4. **Initialization and configuration methods**, such as `init()`, `enableCaching()`,

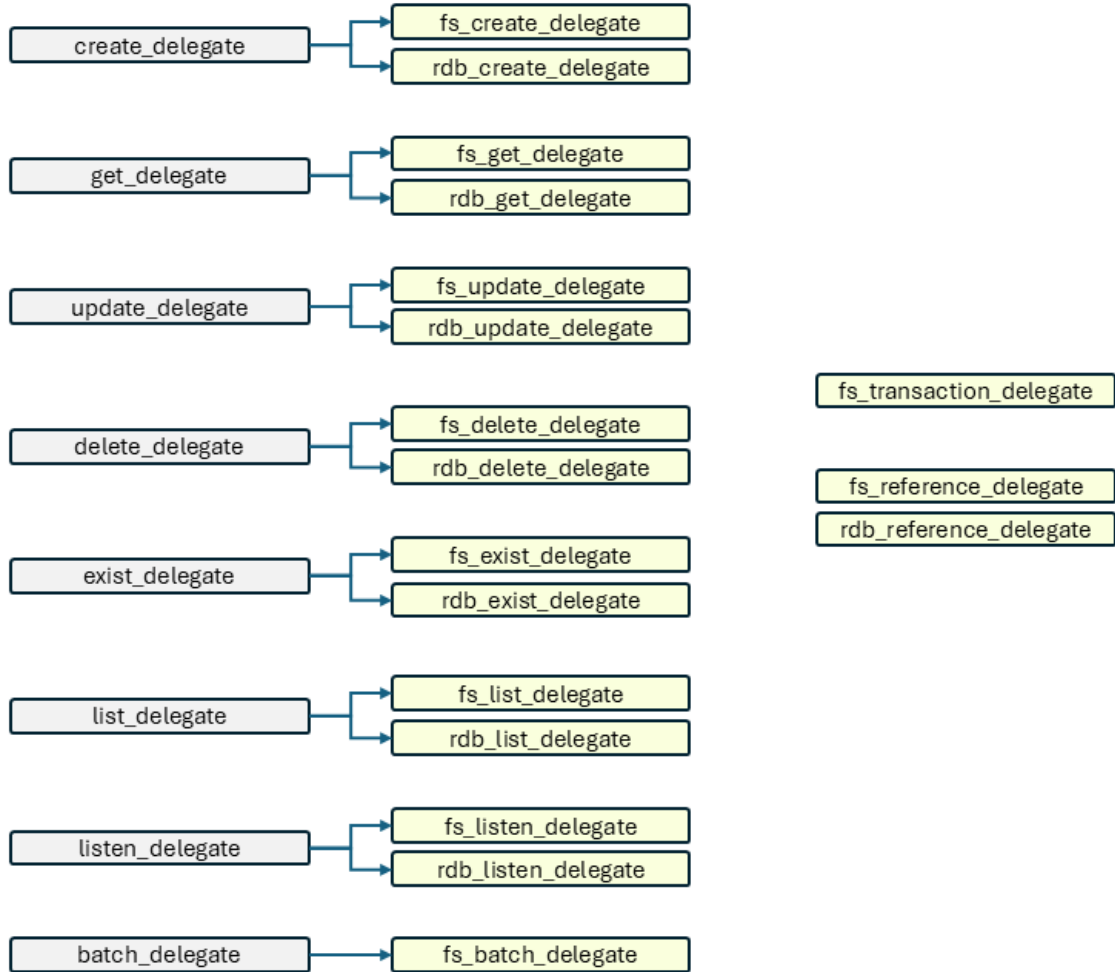


Figure 5.1: A visualization of the delegate abstraction in Firestore.

and so on.

The registry component of Firestore works by integrating registration functions within the data access API with functions which are generated during compile time. These internal structures make it possible for Firestore to determine which data entities are valid, by fetching their serializer or deserializer functions. From the perspective of the developer, such complexities are hidden away. The only step developers must take in this regard is to call `register()`, which automatically registers all compatible classes for use with Firestore.

5.4.1 CRUD operations

Basic database operations such as CRUD are supported by Firestore through the aforementioned delegate structure. These are extended into CRUDLE (Create, Retrieve, Update, Delete, List, Exist) operations in Firestore, as there are frequent use-cases for listing multiple records at a time or simply checking if an item exists.

As an example, the `FSCreateDelegate`, which extends the `CreateDelegate` interface, contains two methods called `one()` and `many()`. The `FS.create.one()` method, shown in Listing 9, receives a `dynamic` object and attempts to find its serializer function based on its type. If a deserializer is not found, the method yields an error informing the developer that this class is not registered. If a matching serializer function is found, the object is serialized and its `id` attribute is extracted. If this attribute is null, the object cannot be written to the database and another error is yielded. Otherwise, the object is created in Firestore by first obtaining a reference to it. In most API functions, there is an optional parameter called `subcollection`. Subcollections enable developers to split data types into subcollections and nest them under the initial collection of documents, enabling faster queries in specific use cases.

The differences between functions created for the Firestore and Realtime database are few, as they both contain the same structure to detect serializers, identify ID fields, and so on. The only difference between the two datastores in most methods, is how a reference is obtained to

```

1  /// Creates a document in Firestore from the given object.
2  @override
3  Future<void> one(dynamic object, { String? subcollection }) {
4      final serializer = FS.serializers[object.runtimeType];
5      if (serializer == null) {
6          throw UnsupportedError('No serializer found for type:
           ↳ ${object.runtimeType}. Consider re-generating Firestore data
           ↳ classes. ');
7      }
8      final map = serializer(object);
9      String id = map["id"];
10     if (id.isEmpty) {
11         throw NullIDException(map);
12     }
13     DocumentReference ref = FS.instance
14         .collection(object.runtimeType.toString())
15         .doc(id);
16     if (subcollection != null) {
17         ref = FS.instance
18             .collection(object.runtimeType.toString())
19             .doc(subcollection)
20             .collection(subcollection)
21             .doc(id);
22     }
23     return ref.set(map);
24 }

```

Listing 9: The `FS.create.one()` method.

```

1  /// Creates a document in the Realtime database from the given object.
2  @override
3  Future<void> one(dynamic object, { String? subcollection }) {
4      final serializer = RDB.serializers[object.runtimeType];
5      if (serializer == null) {
6          throw UnsupportedError('No serializer found for type:
7              ↳ ${object.runtimeType}. Consider re-generating Firestore data
8              ↳ classes.');
```

Listing 10: The `RDB.create.one()` method.

a datastore location, and the functions used to interact with the database. Listing 10 shows the implementation of `RDB.create.one()`, demonstrating the similarities between the two functions.

Several operations involve interactions with multiple objects. In such cases, Firestore attempts to write or read data in the most efficient way. When writing data to the database, the most efficient approach is to bundle everything together into a batch, and send the data together in a single request. This can be seen in the implementations of `FS.create.many()` and `RDB.create.many()` shown in Listings 11 and 12. In such methods, the lack of support for batch operations by the Realtime database is handled by using a custom `RDBWriteBatch`, which bundles data into a map of multiple object updates before sending a single request to the Realtime database.

Operations involving multiple reads differ significantly from those involving multiple writes, as the Firestore and Realtime database SDKs provide access to one database document

```

1 // [...]
2 WriteBatch batch = FS.instance.batch();
3 for (var object in objects) {
4
5     final map = serializer(object);
6     if (map["id"].isEmpty) {
7         throw NullIDException(map);
8     }
9
10    DocumentReference documentReference = FS.instance
11        .collection(T.toString())
12        .doc(map["id"]);
13
14    if (subcollection != null) {
15        documentReference = FS.instance
16            .collection(T.toString())
17            .doc(subcollection)
18            .collection(subcollection)
19            .doc(map["id"]);
20    }
21
22    batch.set(documentReference, serializer(object));
23 }
24 return batch.commit();

```

Listing 11: A fragment of the `FS.create.many()` method.

```

1 // [...]
2 RDBWriteBatch batch = RDBWriteBatch();
3 return batch.create(objects, subcollection: subcollection);

```

Listing 12: A fragment of the `RDB.create.many()` method.

at a time. However, developers may find it cumbersome to make multiple requests manually. Therefore, despite the lack of any cost-efficiency advantages, such functions can still improve developer experience through convenience, as they would enable the retrieval of multiple items simultaneously. As the Firestore and Realtime SDKs limit operations to one at a time, Firestore utilizes concurrent methods to retrieve multiple objects simultaneously. This is done by first creating a list of references to all objects to be retrieved, and then creating multiple asynchronous reads at the same time. The concurrent retrieval of data may benefit developers by reducing the processing time required to retrieve multiple objects. An example of this is demonstrated in Listing 13, where the implementation of `FS.get.many()` is shown. The concurrent execution of multiple read operations and the deserialization of data is achieved through the use of multiple **Futures**, mapping each document reference to its retrieval operation. As soon as the retrieval of all documents is completed, they are deserialized and returned as a list of objects.

Despite already offering performance advantages, such methods could theoretically be improved. In theory, the deserialization of objects could also happen concurrently to reduce processing time. However, a version of this function with concurrency implemented for deserialization was created and compared against the demonstrated version. The results from this test indicated that for a small number of objects the use of concurrency can be detrimental to performance. This could most likely be attributed to the overheads of instantiating various constructs needed to run code concurrently within Dart's Isolates mechanism along with passing data between isolated paths of execution and memory chunks. For such operations, developers are not expected to read large numbers of documents simultaneously, as this would be cost-inefficient and not scalable. Ultimately, it was decided to avoid concurrency for deserialization.

```

1  /// Reads multiple documents from Firestore and converts them to a list of the
   ↪ specified type.
2  /// Sends multiple requests at the same time and returns when all are
   ↪ completed.
3  @override
4  Future<List<T>> many<T>(List<String> documentIDs, { String? subcollection })
   ↪ async {
5      final deserializer = FS.deserializers[T];
6      if (deserializer == null) {
7          throw UnsupportedError('No deserializer found for type: $T. Consider
           ↪ re-generating Firestore data classes.');
```

Listing 13: The implementation of `FS.get.many()`.

```

1  var queryResult = await FS.list.filter<Person>(Person)
2      .orderBy("height")
3      .startAt([1.70])
4      .limit(3)
5      .fetch();
6
7  queryResult.items.forEach((person) {
8      print(person);
9  },);

```

Listing 14: An example use of the `FS.list.filter()` method.

5.4.2 Listing and Queries

While similar, data listing operations differ from those seen in multi-read operations. The primary difference in such methods is that multi-read operations simply retrieve documents based on their IDs, whereas listing operations make use of internal query mechanisms. These provide the ability to limit the number of documents based on query parameters, allowing developers to filter the documents being retrieved. The use of concurrency in such operations could provide advantages, but still remains a work in progress. Developers can use `ListenDelegate` derivatives to list documents of a class with and without a limit. More interestingly, the `filter()` method in each derivative helps developers construct queries through the `FSFilterable` and `RDBFilterable` objects. These objects utilize the Builder pattern to provide a convenient way for developers to stack multiple query clauses on top of each other before making a call to retrieve data. An example of the definition of a query from the developer's perspective is shown in Listing 14.

5.4.3 Advanced features

Transactions and batch operations

Apart from the CRUDLE operations defined above, Firestore supports a few advanced features that make it easier to utilize Firestore and Realtime database features in an object-oriented way. One of these features is the ability to work with objects in Fire-

```

1 FS.transaction.run((transaction) async {
2     Person? p = await transaction.get.one<Person>(person.id);
3     if (p == null) {
4         print("Person not found"); //Error handling logic
5         return;
6     }
7
8     p.age = 20;
9     transaction.update.one(p);
10 });

```

Listing 15: An example use of transactions in Firestore.

store transactions. Object-oriented transactions in Firestore are enabled through the `FSTransactionDelegate` which defines a `run()` function. This function acts as a wrapper around the traditional transaction functionality supported by the Firestore, and defines four transaction delegates: create, get, update, and delete. These transaction delegates are usable within the `FSTransactionHandler`, which is a developer-defined function that defines the steps of the transaction. The handler function makes it easy for developers to carry out transactions by having access to the transaction delegates to carry out various operations as part of that transaction. Listing 15 demonstrates the use of object-oriented transactions from the developer's perspective. In a similar approach, Firestore also supports object-oriented batch operations. In this case, an `FSBatchHandler` needs to be provided by the developer to the `run()` function of the `FSBatchDelegate`, with the ability to use several batch delegates involving create, update, and delete operations.

Real-time listeners

A feature which distinguishes Firestore and the Realtime database from other datastores is their ability to offer bi-directional communication links for real-time data updates. This functionality can enhance application behavior by enabling the development of real-time features such as user presence, live data updates, and more, by attaching listeners to specific items in the database. Firestore provides access to real-time updates on database items

using object instances or ID attributes. Once an object is selected, developers can define handler functions similar to those in transactions and batch writes to handle changes in the documents based on what type of event has occurred. `onCreate()` allows developers to check if an object has been created, `onChange()` detects and allows developers to handle changes to an object, and `onDelete()` notifies that a document being listened to has been deleted. The `handleDocumentListener()` function in each derived `ListenDelegate` is very important to handle this in an object-oriented way, as it maps streams of database snapshots of raw data, which is then deserialized and forwarded to the developer as an object. For each of these converted streams of data, the function `listen()` is called to listen to this object for events. The function, shown in Listing 16, returns a `StreamSubscription`, which can be managed by the developer within the app.

5.5 Demonstrator app

Some of the features outlined in this section are showcased in a Demonstrator app. The demonstrator app was developed as a Flutter mobile application to confirm that Firestore features can be useful in the context of a cross-platform application, as well as to showcase the integration of the package in a real application. The demonstrator app is a simplified contacts application where users can create and explore their list of contacts. It offers minimal functionality, such as a `ListView` which shows contact information such as the name, phone number, email, and address of each contact. Users can create and manage a contact using a simple form, and the app also offers real-time synchronization with Firestore. This means that when data changes in the database, it can be synchronized in real-time through Firestore.

The functionality of the demonstrator app, of which the code is available on GitHub ¹, is shown through screenshots in Figures 5.2a and 5.2b. Users can create a new contact (in

¹Demonstrator app source code: https://github.com/RayLabz/Firestorm/tree/master/firestorm_demonstrator

```

1  /// Converts data from a stream of Firestore documents into a stream of
    ↳ objects.
2  StreamSubscription<T?> _handleDocumentListener<T>(
3      DocumentReference docRef,
4      Deserializer deserializer,
5      Function()? onDelete,
6      Function()? onNull,
7      Function(T object)? onCreate,
8      Function(T object)? onChange,
9  ) {
10     T? previous; //keeps track if this object previously existed.
11
12     //Map stream of snapshots to stream of deserialized objects.
13     final Stream<T?> mappedStream = docRef.snapshots().map((snapshot) {
14         if (!snapshot.exists) {
15             return null; // Document does not exist.
16         }
17         final data = snapshot.data() as Map<String, dynamic>;
18         return deserializer(data);
19     });
20
21     //Listen to the mapped stream and call the appropriate callbacks.
22     return mappedStream.listen((current) {
23         if (current == null) {
24             if (previous != null) {
25                 previous = null;
26                 onDelete?.call();
27             }
28             else {
29                 onNull?.call();
30             }
31             return;
32         }
33
34         if (previous == null) {
35             onCreate?.call(current);
36         } else {
37             onChange?.call(current);
38         }
39
40         previous = current;
41     });
42 }

```

Listing 16: The implementation of `FS.listen._handleDocumentListener()`.

this case ‘Charlie Wilson’) by specifying its details within the form, and then pressing ‘Save contact’. When this happens, `FS.create.one()` is called after basic input validation to create the object in Firestore.

Users can also update contact information by clicking on each contact within the list and changing any of their properties. Once the ‘Update contact’ button is pressed, the updated contact object will be passed to an `FS.update.one()` call, which will update the information in the database. In Figure 5.4a, the user modifies the email address of ‘John Smith’ to have the extension `.comz` instead of `.com`, and this change is immediately visible in the contacts list in Figure 5.4b. It is important to note that the app does not offer a local persistence layer, and that all data is synchronized online to facilitate an inspection of whether Firestore interacts with the database as expected.

Furthermore, contacts can be deleted by pressing ‘Delete contact’ which will call `FS.delete.one()` on that contact object. As seen in Figure 5.4, the contact ‘John Smith’ is deleted from the list of contacts when the delete button is pressed.

5.6 Deployment & Guide

The Firestore package is published as a Beta and is publicly available on pub.dev ², which is the primary package repository for Dart and Flutter. At the time of writing this thesis, the package has been downloaded about 300 times. The code for Firestore for Flutter is open-source and is hosted on GitHub under the Apache 2.0 License ³.

A guide for developers on how to use Firestore is also available on pub.dev and its GitHub repository. The guide provides an overview of what is Firestore, and explains its objectives and features. More importantly, it lists several steps to get developers started with using Firestore:

²Firestore on pub.dev: <https://pub.dev/packages/firestorm>

³Firestore for Flutter on GitHub: https://github.com/RayLabz/Firestorm/tree/master/firestorm_flutter

12:53

← Add Contact

Firstname
Charlie

Lastname
Wilson

Email
CWilson@example.com

Phone
+1-555-2217778

Address
Street
Eighth St

City
Hill Valley

Save Contact

DEBUG

(a) Creating a contact in the demo app.

12:54

Contacts

J John Smith
JSmith@example.comz
+1-555-9552044

H Hannah Wilson
HWilson@example.com
+1-555-510049

G George Smith
GSmith@example.com
+1-555-9275072

F Fiona Jones
FJones@example.com
+1-555-7604239

C Charlie Wilson
CWilson@example.com
+1-555-2217778

Contact saved successfully

DEBUG

(b) List of contacts in the demo app.

Figure 5.2: Creating and saving a contact using Firestore in the demo app.

12:54

← John Smith

Firstname
John

Lastname
Smith

Email
JSmith@example.comz

Phone
+1-555-9552044

Address
Street
Sixth St

City
Cypress Creek

Update Contact Delete contact

(a) Updating a contact in the demo app.

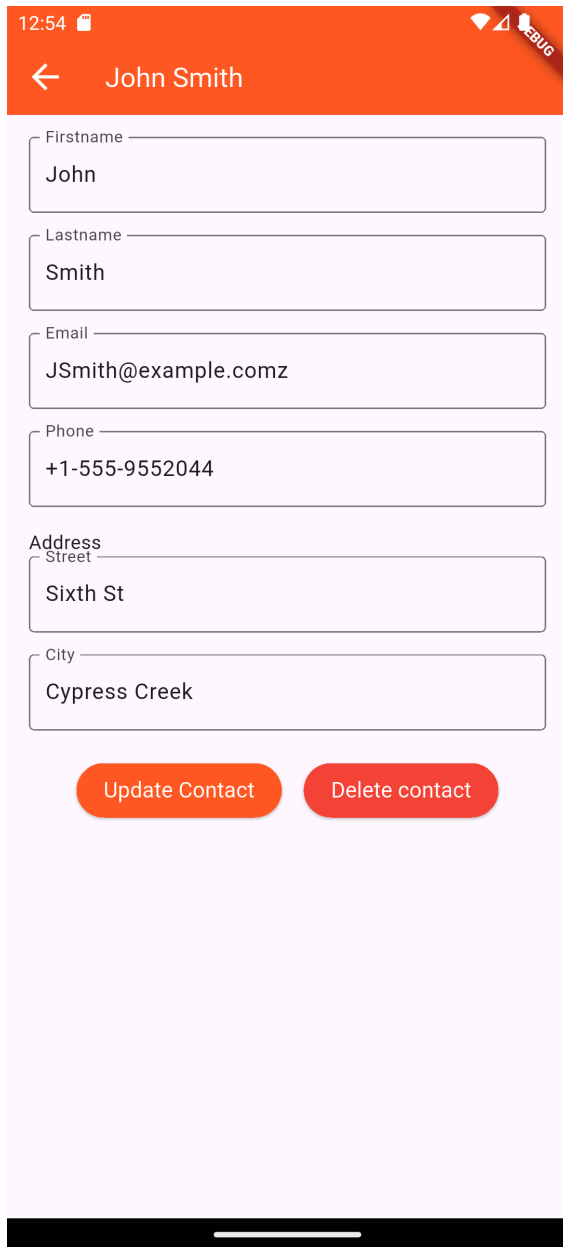
12:53

Contacts +

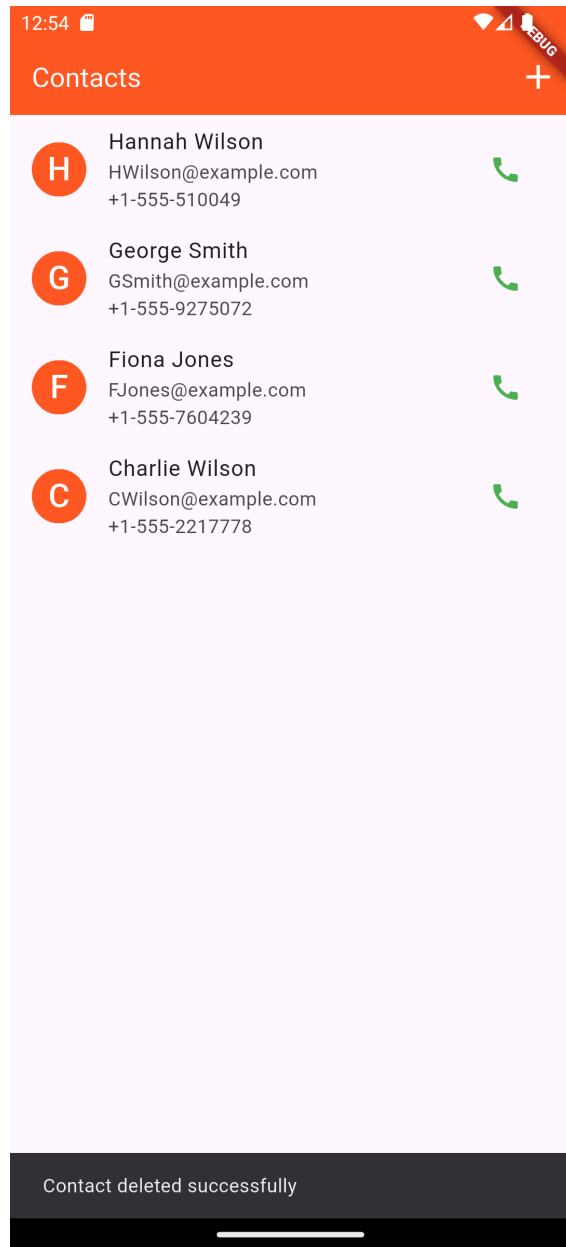
J	John Smith JSmith@example.comz +1-555-9552044	📞
H	Hannah Wilson HWilson@example.com +1-555-510049	📞
G	George Smith GSmith@example.com +1-555-9275072	📞
F	Fiona Jones FJones@example.com +1-555-7604239	📞

(b) Updated list of contacts in the demo app.

Figure 5.3: Updating a contact in the demo app.



(a) Deleting a contact in the demo app.



(b) Updated list of contacts in the demo app.

Figure 5.4: Deleting a contact in the demo app.

1. **Installation:** The steps involved in installing Firestore.
2. **Defining data entities:** How to define a data entity that can be used by Firestore, the accepted data types, and rules to be followed.
3. **Model generation:** How to use `build_runner` to generate the necessary code for Firestore.
4. **Initializing Firestore:** How to initialize Firestore by running the `init()` and `registerClasses()` functions.
5. **Importing and using Firestore:** How to import Firestore and run operations, according to the API Guides.

The Firestore API guide is also instrumental in helping developers use the data access API. The API guide is divided into the Firestore API Guide⁴ and the Realtime Database API Guide⁵, both of which feature various examples on how to run all types of operations supported by each database. The guide also provides guidelines for developers regarding performance considerations⁶ to help them maximize efficiency when dealing with Firebase datastore operations. Finally, some evaluation data is also available within the same page, allowing developers to view an empirical evaluation of Firestore in comparison with the native APIs and determine its performance overheads.

A full API reference is also available at `pub.dev`⁷.

⁴Firestore Firestore API Guide: https://github.com/RayLabz/Firestorm/blob/master/firestorm_flutter/api-docs/fs-api/fs-api.md

⁵Firestore Realtime database API Guide: https://github.com/RayLabz/Firestorm/blob/master/firestorm_flutter/api-docs/rdb-api/rdb-api.md

⁶Performance considerations and comparison: https://github.com/RayLabz/Firestorm/blob/master/firestorm_flutter/performance.md

⁷Full API reference: <https://pub.dev/documentation/firestorm/latest/>

5.7 Summary

This section described the internal workings of Firestorm, including explanations of various implementations and challenges. The structure of the package and various code components have been presented through various listings and figures, explaining the functionality of the important elements in the code. The section also presented the demonstrator app, used to showcase the integration of Firestorm into a Flutter application, and discussed the deployment of the package along with details about its guide.

Chapter 6

Testing

6.1 Introduction

This chapter discusses the testing objectives and presents the approaches and tools used during the testing process.

6.2 Testing objectives

Firestorm was tested to determine the correctness of the tool and confirm its compatibility with Flutter application development. The *verification* of Firestorm code is essential to ensure that the implementation of this tool has correctly addressed the requirements specified in Tables 3.2 and 3.3. The testing objectives are as follows:

1. Verify the correctness of the code analysis and generation components.
2. Confirm the correctness of data operations in both datastores.
3. Ensure the reliability of real-time data updates

Further evaluation-specific objectives are considered and discussed in chapter 7, according to the research questions defined in section 1.3.

6.3 Testing platform

Testing was undertaken by creating and utilizing a Flutter project as the testing platform. The testing platform incorporated manually executed unit tests, along with integration tests using the `integration_test` package. These were imported to provide the necessary tools to run test code. The integration testing code is located under the `integration_test` folder and is broken into different files based on the operations and tools being tested. Android Studio and the Android Emulator were used to carry out testing on top of an emulated Android operating system. The Firebase Emulator suite was also used to conduct testing for each respective datastore, which was configured by creating rules and other configuration options for Firestore and Realtime database.

6.4 Unit Testing

Despite making several attempts to conduct automated unit testing on the entity analysis and code generation tools using `flutter_test` and `mockito`, this was not possible due to the compile-time nature of these tasks. Such testing automation tools are not compatible with tools running at compile time, such as the `analyzer` package which is responsible to provide class inspection features for entity analysis. As a result, unit testing was done manually by first defining unit tests, running them, and then comparing their expected and computed outputs. Table A.1 shows a selection of the predefined unit test cases which were used during this testing process.

The testing process revealed several bugs, such as inconsistencies during the code generation process which resulted in code which did not compile. In several cases, generated code which did compile resulted in unexpected null values due to incorrect code generation for mapping object data to map entries and vice versa, especially when it came to composite objects. The testing process also revealed the need to offer support for named parameters as more advanced versions of entities were introduced. In this case, the initial version of

the analyzer and generator code did not detect named parameters at all, which resulted in empty mappings when an entity constructor only contained named parameters.

6.5 Integration Testing

Unit testing primarily aimed to verify the behavior of compile-time stages such as entity analysis and code generation. However, this testing approach is not helpful in covering any runtime functionality associated with datastore operations. To verify runtime functionality, integration tests are necessary as the Firestore and Realtime database SDKs are not compatible with unit testing in Flutter. The primary objective of integration testing is to verify the behavior of data operations across both Firestore and Realtime database, including basic CRUDLE operations, real-time listeners, batch writes, transactions, and pagination. The code created for integration tests utilizes the test cases presented in Table B.1, and is part of the `integration_test` folder. This code is divided into two files, each meant to test data operations for each of the two datastores. Such tests can be executed by running `flutter test integration_test` in the terminal while in the testing project folder.

Integration tests were useful in identifying a plethora of issues which went undetected during the development process. For instance, integration testing resulted to the identification of a serious bug in transaction logic, which arose from the fact that multiple `transaction` objects were being utilized within the code. Firebase, unlike many other frameworks, does not use lock-based transactions and instead relies on serialization-based transactions. In this approach, transactions do not lock resources on the database, but rather utilize document timestamps to track if a document has changed since it was last accessed by the transaction. Hence, each transaction can be idempotent, but only within the scope of the transaction itself and not for other transactions. Such a profound observation would not have been possible had it not been for integration tests which attempted to test the behavior of object-oriented transaction operations supported by Firestore. Various other integration tests also helped

```

1 testWidgets("Test paginate()", (tester) async {
2   await FS.delete.all(ComputingStudent, iAmSure: true); //Setup
3   await FS.create.many(students); //Setup
4
5   var paginator = FS.paginate<ComputingStudent>(limit: 3);
6
7   var currentPage = await paginator.next();
8   assert(currentPage.items.length <= 3);
9
10  while (currentPage.items.isNotEmpty) {
11    currentPage = await paginator.next();
12    assert(currentPage.items.length <= 3);
13  }
14 });

```

Listing 17: An integration test, testing the behavior of `FS.paginate()`.

with the verification of the behavior or several operations, ultimately leading to increased confidence in the usefulness of the package.

Various listings demonstrate the use of integration testing within the testing platform. Firstly, Listing 17 includes an integration test used to verify pagination behavior from `FS.paginate()`, (Test 7a in B.1). In this test, the code creates a new paginator object, and then iteratively attempts to retrieve pages, ensuring that the contents of each page meet the pre-defined limit. Furthermore, Listing 18 demonstrates two variations of integration tests for the Realtime database which test the function `RDB.list.ofClass()` with different values for the limit parameter. The Firestore transaction integration test is also shown in Listing 19, where an object-based transaction is carried out through Firestore. The test runs the transaction, which changes an attribute of the object specified and then confirms that once the transaction is completed, the attribute has changed in the database. Source code for all integration tests is available on GitHub under the test platform project ¹.

¹Integration tests: https://github.com/RayLabz/Firestorm/tree/master/flutter_test_app/integration_test

```

1 testWidgets("Test list.ofClass() without limit", (tester) async {
2     List<ComputingStudent> result = await RDB.list.ofClass(ComputingStudent);
3     assert(result.length == students.length);
4 });
5
6 testWidgets("Test list.ofClass() with limit", (tester) async {
7     List<ComputingStudent> result = await RDB.list.ofClass(ComputingStudent,
8         ↪ limit: 3);
9     assert(result.length <= 3);
10 });

```

Listing 18: An integration test, testing the behavior of `RDB.list.ofClass()`.

```

1 testWidgets("Transaction test", (tester) async {
2     await FS.delete.all(ComputingStudent, iAmSure: true);
3     await FS.delete.one(student);
4     await FS.create.one(student);
5
6     FS.transaction.run((transaction) async {
7         ComputingStudent? s = await
8         ↪ transaction.get.one<ComputingStudent>(student.id);
9         if (s != null) {
10             student.firstname = "CHANGED!";
11             transaction.update.one(student);
12         }
13     }).then((value) {
14         FS.get.one<ComputingStudent>(student.id).then((result) {
15             assert(result != null);
16             if (result != null) {
17                 assert(result.firstname == "CHANGED!");
18             }
19         },.);
20     },.);
21 });

```

Listing 19: An integration test, testing the behavior of object-oriented transactions in Firestore.

6.6 Compatibility Testing

The aim of this project is to provide Firestorm for use in various major platforms, including Android, iOS, Web, MacOS, and Windows (Firestore-only). However, due to time limitations, it was only possible to confirm the compatibility of the package with the Android ecosystem. While the cross-platform nature of the Flutter framework makes it easy for such tools to be integrated with a variety of platforms, further compatibility testing is needed in the future to confirm that Firestorm works as expected in all of these platforms.

6.7 Summary

This section describes the testing strategy and objectives, along with the tools utilized to run several unit and integration tests. Several unit and integration test cases are presented, along with listings of some integration tests. Overall, the testing process has ensured that Firestorm functions correctly and without serious defects. Despite this, further exploration is necessary to evaluate several other aspects of Firestorm, such as its performance, maintainability, and feature coverage, which are discussed in the next chapter.

Chapter 7

Evaluation

7.1 Introduction

This chapter presents an evaluation of Firestorm, which is based on three components: determining its performance overheads, the level of code maintainability, and a comparative analysis of its feature coverage. This evaluation is conducted using several steps, which were initially introduced in section 3.7, and are further expanded on in this chapter. Each of the evaluation components are aligned with the research questions identified in section 1.3, aiming to assess the effectiveness and usefulness of Firestorm toward maintainable cross-platform application development.

7.2 Performance overheads

The first component focuses on measuring the processing time overheads introduced by Firestorm, in direct comparison with the performance of the native Firebase APIs. As described in section 3.7, all tests were conducted under a controlled environment to ensure the consistency and reproducibility of the results obtained. The testing platform was used to write the evaluation code through the creation and use of an *evaluation harness*. The harness is responsible for running different parts of the evaluation selectively, allowing the

execution of different parts of code of which the performance needs to be compared.

The performance of the code is determined by measuring the processing time in milliseconds. This is defined as the time taken for a specific set of statements specifically related to the operation being tested to be executed, received, and persisted by the database, and a response to be received at the client-side. In this evaluation only the main operations are compared, such as creating, retrieving, updating, and deleting items, along with their multi-object counterparts. This is done for both the Firestore and Realtime database APIs for Firestorm and the native SDKs correspondingly. In detail, the following operations are evaluated:

- Single object create (`FS.create.one()` and `RDB.create.one()`).
- Single object get (`FS.get.one()` and `RDB.get.one()`).
- Single object update (`FS.update.one()` and `RDB.update.one()`).
- Single object delete (`FS.delete.one()` and `RDB.delete.one()`).
- Multi-object create (`FS.create.many()` and `RDB.create.many()`).
- Multi-object get (`FS.get.many()` and `RDB.get.many()`).
- Multi-object update (`FS.update.many()` and `RDB.update.many()`).
- Multi-object delete (`FS.delete.many()` and `RDB.delete.many()`).

7.2.1 Measurement procedure

The processing time for each operation is measured using Dart's **Stopwatch** tool, which is a high-resolution timer. For each of the operations listed above, three trials are executed and averages are calculated to improve the reliability of the results. For instance, in single-object operations, three different trials are executed, each consisting of 100 synchronously executed reads (one after each other). To ensure this, the calls to these operations are made

synchronously using `await`, and a delay of 100 milliseconds is added between operations as a padding to allow adequate time for Firestore to assimilate data and return to an idle state. For multi-object operations, each trial consists of just a single operation which operates on 100 objects, with three trials recording the time taken for this operation to be completed. The trials described above are meant to measure the processing time of these operations with a constant set of objects. During each run, the code automatically instantiates only the necessary data needed in these operations, which is subsequently deleted from the database after each test to avoid inconsistencies due to database load. The code for this setup is available on the project's GitHub repository ¹.

7.2.2 Results

The complete set of results presented in this section is available online ².

Firestore

Table 7.1 compares the average processing time of single object operations in Firestore between the native API and Firestorm, which are also visualized in Figure 7.1. As the results show, Firestorm's `FS.create.one()` operation marginally outperforms its Native API counterpart, whereas the rest of the single-object operations perform marginally worse than the Native API with `FS.update.one()` performing relatively worse with about 3% added processing time compared to the native API. However, the performance of all single-object operations in Firestore are well within the threshold margin of +10% increase in processing time.

The results for multi-object operations are shown in Table 7.2, and visualized in Figure 7.2. From these, we can see that the `FS.create.many()` operation offers substantially better performance compared to its native API counterpart, with 17% less processing time.

¹Evaluation code: https://github.com/RayLabz/Firestorm/tree/master/flutter_test_app/lib/evaluation

²Results: https://msuclanac-my.sharepoint.com/:f/g/personal/nkasenides2_uclan_ac_uk/Eo_MgCgXMrNApUZvh6CPRTkBk-H-UjGtsLS_1502P9mIGw?e=pAXdxs

	Native API (ms)	Firestore (ms)	Difference	Threshold
FS.create.one()	32.90	32.54	-1.09%	Pass
FS.get.one()	57.13	57.67	+0.94%	Pass
FS.update.one()	42.13	43.47	+3.18%	Pass
FS.delete.one()	45.88	45.89	+0.03%	Pass

Table 7.1: Comparison of Firestore Native API and Firestore performance across single object operations.

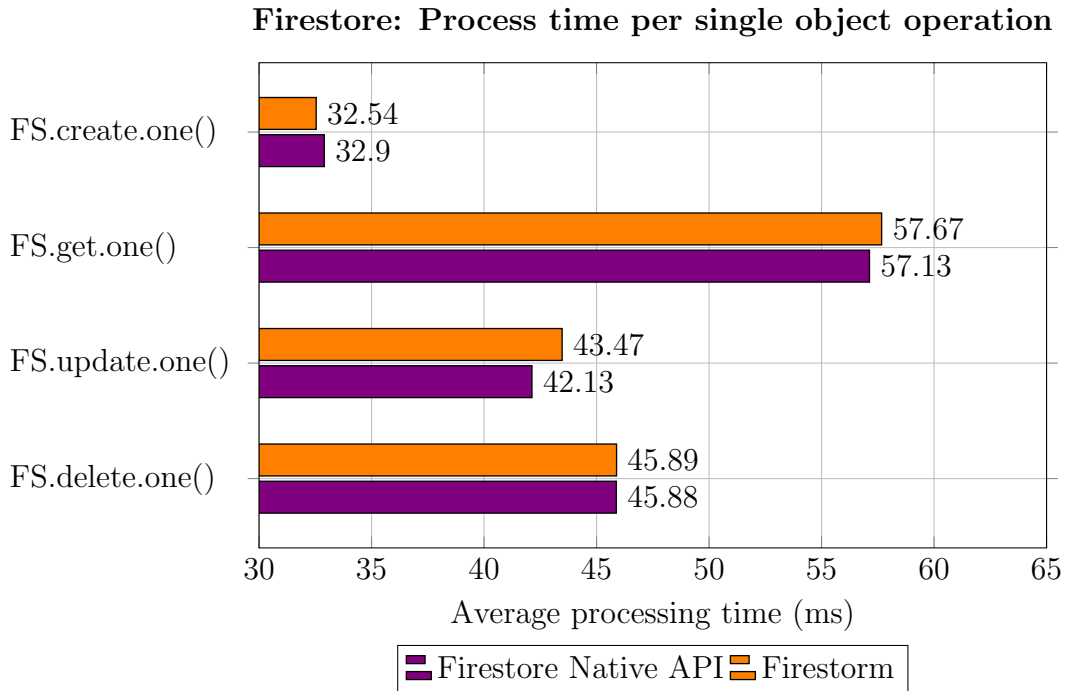


Figure 7.1: Average processing times for single-object operations in Firestore Native API and Firestore

Meanwhile, the `FS.get.many()` and `FS.delete.many()` methods are on-par with their native API counterparts in terms of processing time and perform better by an insignificant amount. More interestingly, the `FS.update.many()` method seems to provide a significant improvement of 95% in processing speed over its corresponding Firestore API call. While the reasons for this are not yet fully understood, evidence from single-object operations suggests that update operations are overall more efficient than other types of operations in Firestore. As such, this method may benefit from taking advantage of multi-object write batches that are significantly more efficient at larger object counts but this remains open

for further exploration.

	Native API (ms)	Firestore (ms)	Difference	Threshold
FS.create.many()	184.83	153.56	-16.92%	Pass
FS.get.many()	4772.47	4762.10	-0.22%	Pass
FS.update.many()	4481.75	219.31	-95.11%	Pass
FS.delete.many()	4883.64	4863.12	-0.42%	Pass

Table 7.2: Comparison of Firestore Native API and Firestore performance for multi-object operations.

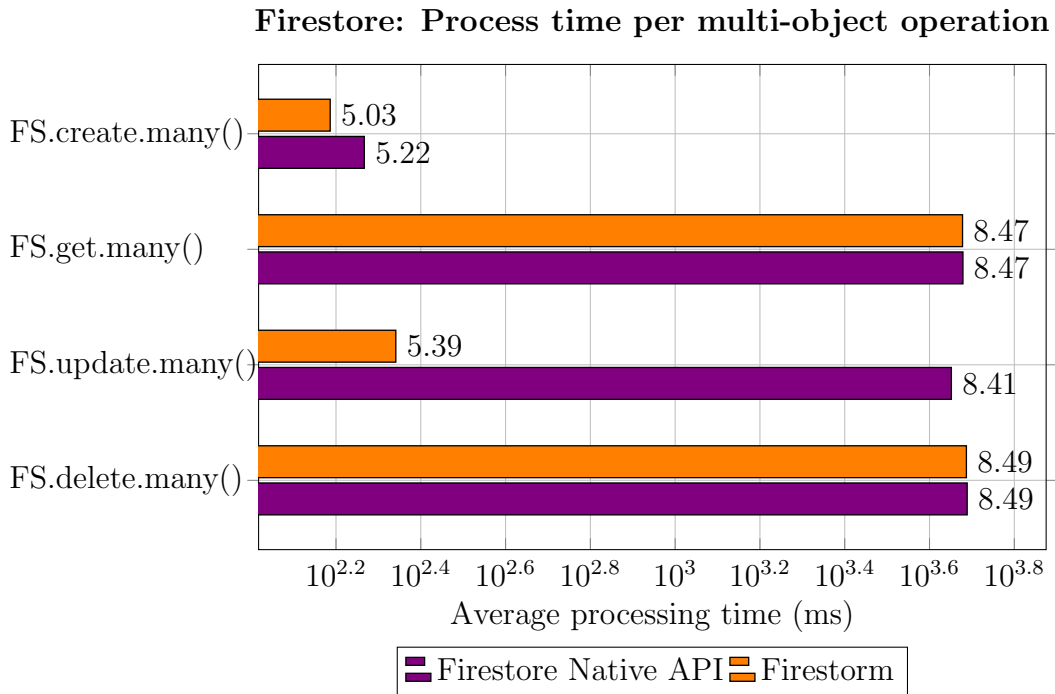


Figure 7.2: Average processing times for multi-object operations in Firestore Native API and Firestore

Realtime database

The Realtime database operations are also evaluated in an identical way. The average processing times for each single-object operation for the Realtime database are summarized in Table 7.3 and shown visually in Figure 7.3. The results show that the `RDB.create.one()` and `RDB.update.one()` operations introduce a 5.6% and 1.5% overhead on processing

time correspondingly. At the same time the `RDB.delete.one()` method makes an insignificant performance impact, whereas `RDB.get.one()` enjoys a slight performance improvement over its native API counterpart. All methods, including the single-object create one which introduces some overhead, are below the maximum overhead threshold of +10%.

	Native API (ms)	Firestore (ms)	Difference	Threshold
<code>RDB.create.one()</code>	5.75	6.07	+5.57%	Pass
<code>RDB.get.one()</code>	5.15	5.00	-2.91%	Pass
<code>RDB.update.one()</code>	4.87	4.94	+1.44%	Pass
<code>RDB.delete.one()</code>	5.04	5.03	-0.20%	Pass

Table 7.3: Comparison of Realtime database Native API and Firestore performance for single-object operations.

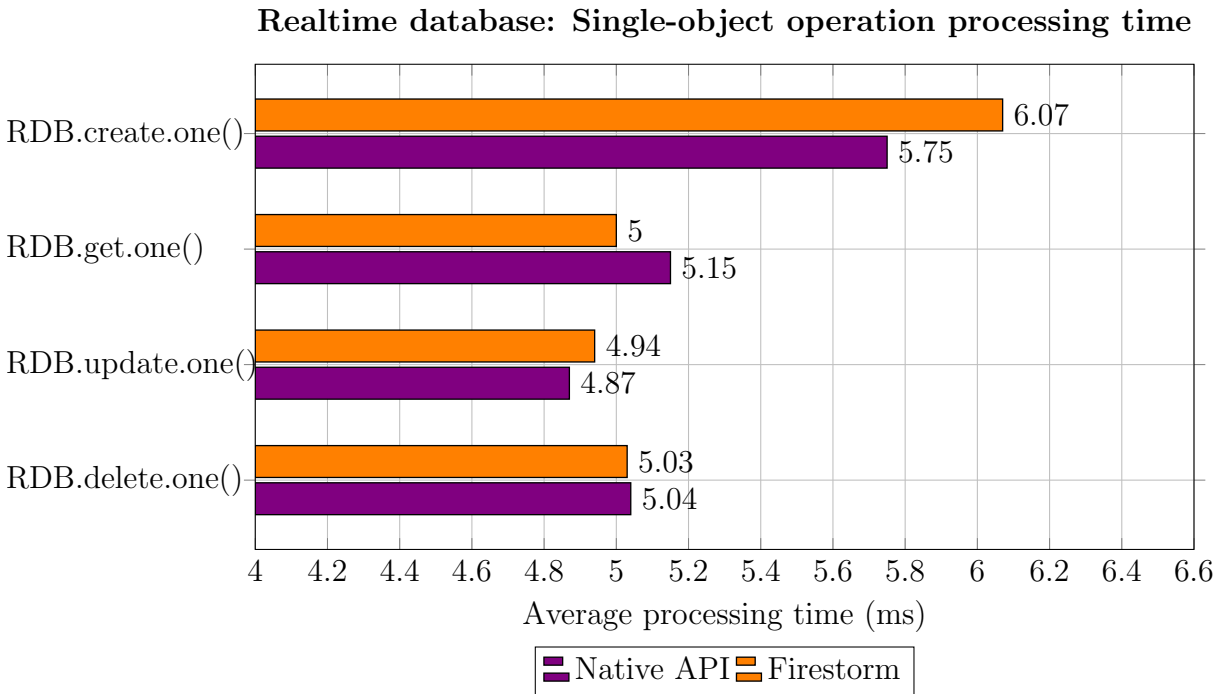


Figure 7.3: Average processing times for single-object operations in RDB Native API and Firestore

Finally, multi-object operations in Realtime database were also tested. The results for these are summarized in Table 7.4 and visualized in Figure 7.4. These results provide evidence that Firestore offers significant advantages for multi-object operations in Realtime database.

The `RDB.create.many()`, `RDB.update.many()`, and `RDB.delete.many()` functions all enjoy a major performance improvement ranging from 84% to 92% compared to the native API functions. At the same time, the `RDB.get.many()` operation is on-par with the corresponding operation in the native API, suffering from a negligible performance overhead. With regards to multi-object operations in Realtime database, this data paints a clear picture: Firestore not only stays below the performance overhead threshold, but also provides significant improvements over the native Realtime database API.

	Native API (ms)	Firestore (ms)	Difference	Threshold
<code>RDB.create.many()</code>	187.25	29.75	-84.11%	Pass
<code>RDB.get.many()</code>	159.47	160.32	+0.53%	Pass
<code>RDB.update.many()</code>	187.81	15.86	-91.56%	Pass
<code>RDB.delete.many()</code>	163.35	13.13	-91.96%	Pass

Table 7.4: Comparison of RDB Native API and Firestore performance for multi-object operations.

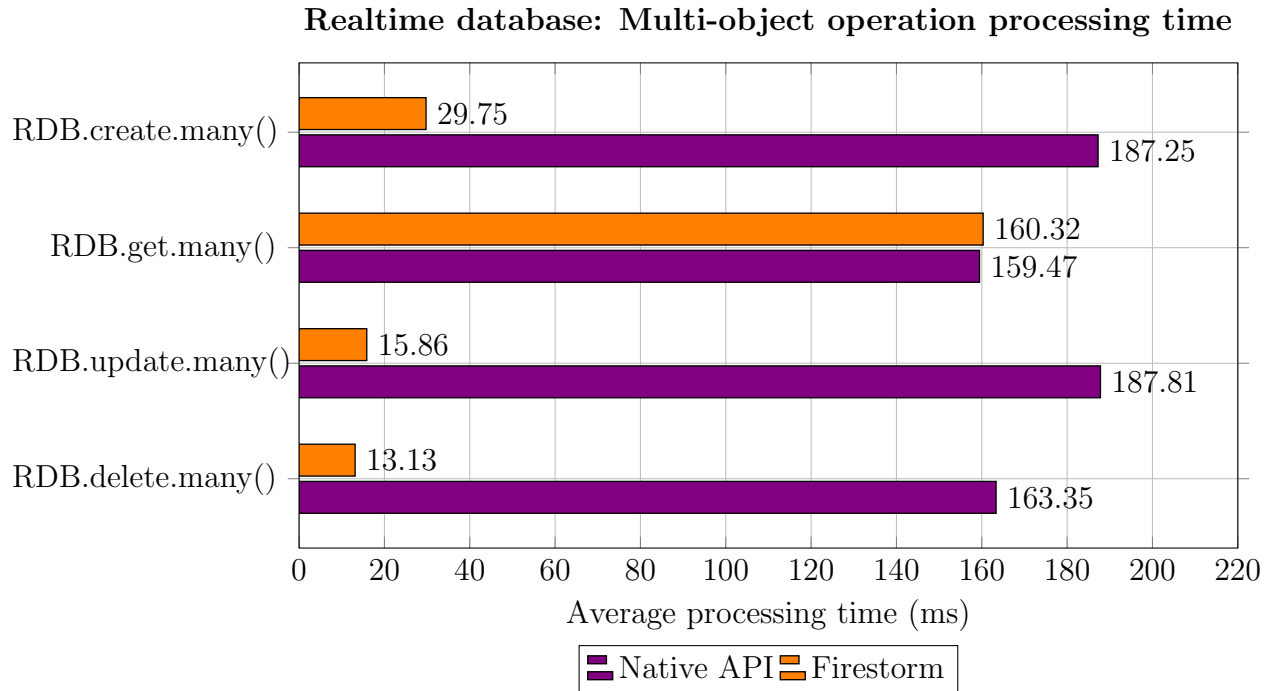


Figure 7.4: Average processing times for multi-object operations in RDB Native API and Firestore

7.2.3 Discussion

The results obtained from the performance evaluation can be used to answer the third research question (**RQ3**): “*What are the performance overheads introduced by Firestore compared to the Firebase SDKs, in terms of processing time?*”

To answer this question, the results are analyzed in two ways. In the first, the percentage difference in performance is used to categorize each operation’s overhead status into one of the following categories, using the following boundaries:

- **Significant overhead:** Above +10% increase in processing time
- **Slight overhead:** Between +10% and +5% difference in processing time
- **Negligible overhead:** Between +5% and >0% difference in processing time
- **Parity:** No (0%) difference in processing time
- **Negligible improvement:** Up to -5% decrease in processing time
- **Slight improvement:** Between -5% and -10% decrease in processing time
- **Significant improvement:** Over -10% decrease in processing time

Figure 7.5 shows the number of operations per category for Firestore. Three of these operations had negligible performance overhead, another three operations had negligible improvement, and two of operations made significant improvements in performance. This shows that for most operations in Firestore, the overhead or improvement in terms of performance was negligible, and likely subject to statistical error.

Figure 7.6 shows the same distribution per category for the Realtime database. In this case, three operations enjoyed significant improvement in performance, two enjoyed a negligible improvement, another two had a negligible overhead, and one suffered from a slight overhead. This data points out that the majority (5 out of 8) of operations enjoyed a performance improvement, whereas the rest of them suffered from a performance overhead. Importantly,

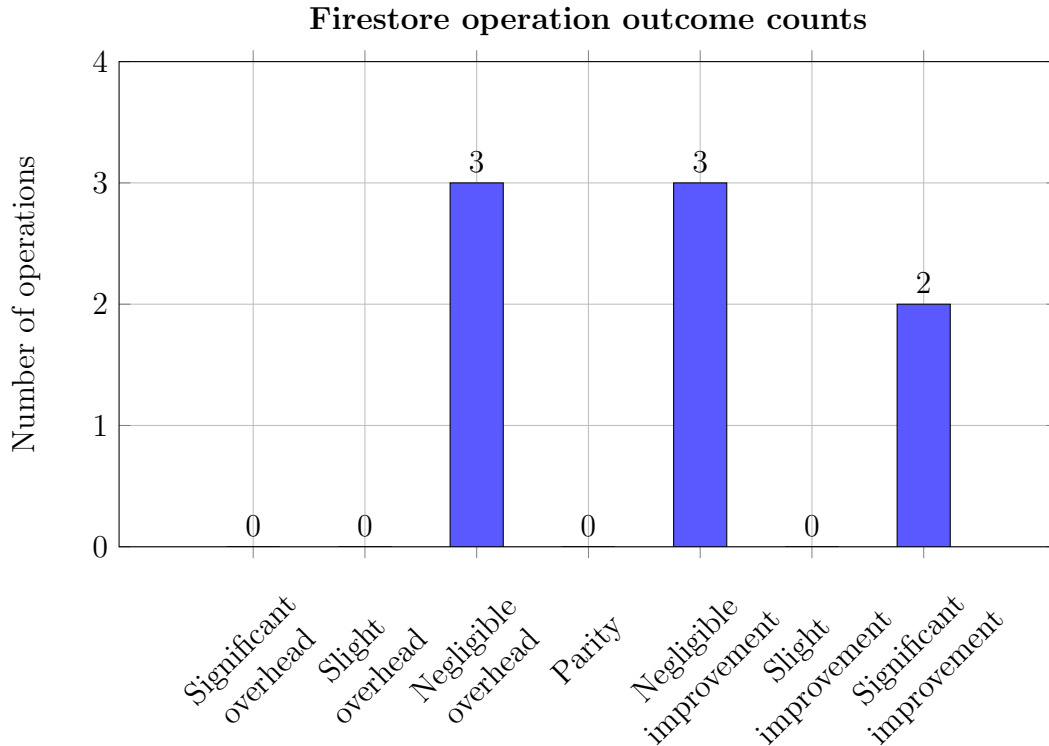


Figure 7.5: Firestore: Count of operations per performance category.

the category with the most frequency in this case is the one of significant improvement, highlighting the fact that Firestore provides a significant advantage in performance.

For both datastores combined, we can see that 5 operations had significant improvement, 5 had negligible improvement, 5 had negligible overhead, and 1 had a slight overhead. Based on this, most operations (10 out of 16) provided an improvement in performance, whereas 6 of them suffered from overhead.

The second way used to analyze the data is by averaging the differences between Native APIs and Firestore among all operations. For the Firestore, this difference averages at -13.7%, which indicates a performance increase overall. The average difference is even higher for the Realtime database at -32.9%, indicating an even stronger performance increase. Combined, the average difference across all operations, across both datastores is -23.3%, which is a very positive outcome given that Firestore adds new layers of software on top of those used to interact with Firebase.

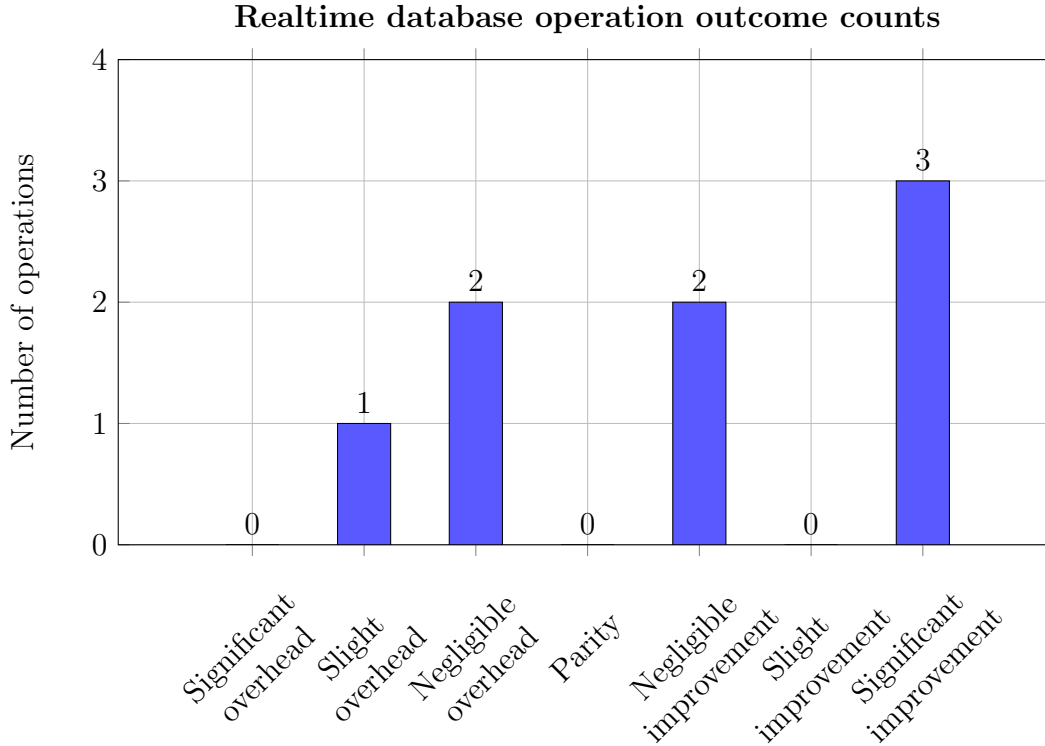


Figure 7.6: Realtime database: Count of operations per performance category

The presence of so many operations with improvement in performance is surprising, as the expectation was that there would be at least a slight overhead in nearly all of the operations tested. However, the design strategies used to build Firestore as well as programming techniques employed to reduce such overheads may have played a significant role in improving performance. Despite these positive results, the answer to the research question posed depends on the identification of the methods which did suffer from performance overheads. Based on this data, the worst performing operation was Realtime database’s single object write, which is perhaps one of the most used operations. Despite this being only a slight overhead, the frequency of use of single object writes means that it is important to revisit the implementation of this method to identify ways to reduce these overheads if possible. Other methods, such as Firestore’s single object retrieval, update, and delete are considered far less important, as their difference in performance is very close to 0, and statistically insignificant.

7.3 Code maintainability

The second aspect of the evaluation aims to assess the maintainability of the code provided by the data access API. As discussed in section 3.7, the code’s maintainability is measured using the Maintainability Index (MI), which is a popular maintainability metric. The MI is a composite metric of three other metrics: Lines of Code, Halstead Volume, and Cyclomatic complexity, as shown in Equation 3.7.2.

7.3.1 Measurement procedure

Similar to the performance evaluation, the experimental setup makes use of the testing platform to host testing code. For this evaluation, a set of selected operations for Firestore was chosen as a sample. There are two reasons why only Firestore operations were chosen for this aspect of the evaluation. Firstly, the data access APIs of Firestore and the Realtime database are nearly identical (Firestore API is a superset of the Realtime database API), and secondly, the Firestore API enables more functionality to be tested, such as pagination and transactions, which are not supported by the Realtime database. Each operation was implemented in two snippets: one using the Firestore native API and one using Firestore’s API. As both native and Firestore APIs support both blocking and non-blocking modes, both of these modes were tested for each operation selected. Where needed, versions of these functions were also tested using error checking and without error checking.

The code implemented in these snippets is syntactically correct but not necessarily compilable. The purpose of this evaluation is not to run the code, but simply measure the maintainability index of the code through code analysis. This is done by using an LLM-powered (Large Language Model) AI agent approach, using ChatGPT’s o4-mini model. Firstly, the model’s ability to identify code tokens and constructs in the Dart programming language was confirmed by dry-running a series of snippets to confirm the validity of the results (i.e., confirm the validity of weights attached to certain constructs, function calls, etc.). After

this, the model was given one snippet of code at a time and was asked to count the non-empty lines of code, determine the cyclomatic complexity, calculate and Halstead volume of the snippet. Finally, the model tabulated the results, which were used to make comparisons between snippets written for the same operation. Listing 20 illustrates an example of two pieces of code written for the native and Firestore API correspondingly. In this case, the pagination functionality is tested with a single page retrieval, in blocking mode, and without error checking in place. It should be noted that when using the native API, serialization and deserialization functions generated by Firestore are used. In this case, it is assumed that developers would manually implement such functions themselves. While this provides an advantage to the native API approach, measuring the impact of producing this code for the native operations is beyond the scope of this test, although it is clear that undertaking such an activity would increase the overall time taken to write code, and the chances of the developers introducing errors. While this is important for maintainability, measuring the effects of this downside is beyond the scope of this evaluation.

7.3.2 Results

The results presented in Table 7.5 are organized into test groups, where each test group contains multiple tests related to one operation, in blocking, non-blocking, and error handling versions. The results of these tests are averaged per test group to determine the overall MI for that operation, where higher MI is preferred.

For three of the single-object operations (create, delete, update), the maintainability indices of the Firestore API and Firestore are identical at 76 and 78 respectively, indicating a slight maintainability improvement of 2.56% in each. For the single-object retrieval operation group (GET_ONE), this improvement is much more significant, at 22.47%. In the multi-object operation group (CREATE_MANY), the Firestore API scored a 67.7 MI, whereas Firestore scored 78.3 — a significant increase in MI (13.62%). Several more advanced operations were also tested, such as Pagination, Queries, Real-time listeners, and Transactions.

```

1 //TEST PAGINATION_3 - Firestore API - Pagination, Blocking, no error checking
2 QuerySnapshot snapshot = await
  ↳ fs.collection('Student').orderBy('name').limit(10).get();
3 if (snapshot.docs.isNotEmpty) {
4   List<ComputingStudent> students = snapshot.docs.map((doc) =>
5     ↳ ComputingStudent.fromMap(doc.data() as Map<String, dynamic>)).toList();
6   print('Students retrieved successfully: $students');
7 } else {
8   print('No students found');
9 }
10
11 //-----
12 //TEST PAGINATION_3 - Firestore FS - Pagination, Blocking, no error checking
13 FSQueryResult<ComputingStudent> result = await
14   ↳ FS.paginate<ComputingStudent>().next();
15 if (result.items.isNotEmpty) {
16   print('Students retrieved successfully with Firestore FS: ${result.items}');
17 } else {
18   print('No students found with Firestore FS');
19 }

```

Listing 20: The pagination function used for MI analysis, in blocking mode, without error checking.

Test group	Firestore API	Firestore	Difference
CREATE_ONE	76	78	2.56%
CREATE_MANY	67.667	78.333	13.62%
DELETE_ONE	76	78	2.56%
GET_ONE	58.667	75.667	22.47%
UPDATE_ONE	76	78	2.56%
PAGINATION	54.333	61.667	11.89%
QUERY	50.667	56.667	10.59%
REAL_TIME_LISTENERS	60	66	9.09%
TRANSACTION	52	59	11.86%

Table 7.5: Comparison of Maintainability Index for the Firestore API and Firestore operation.

For pagination and transactions, the improvement in MI was about 12%, in queries about 10.5%, and for real-time listeners this difference was slightly lower at about 9%. These test groups can be further categorized in single-op, multi-op, and complex-op meta-groups, with the purpose of determining the MI impact on these types of operations more generally. From

this we can see that the most significant improvement in MI stems from multi-op operations (average: 13.62%), followed by complex ops (average: 10.82%), with the single-op group impacted the least with an average MI difference of 7.43%. Among all operations tested, the average MI index difference between the Firestore API code and Firestore code indicates that Firestore code is 10.62% more maintainable compared to native API code.

7.3.3 Discussion

The results obtained from this experiment can be used to answer the first research question (RQ1): *“To what extent does using Firestore improve code maintainability in developed applications compared to using native Firebase SDKs?”*

The data shows that Firestore provides measurable improvements in maintainability across all of the operation categories. The single-object operations demonstrated slight gains, which suggests that even in simple scenarios where code complexity is limited, the abstraction layer introduced by Firestore contributed positively to maintainability. The most significant impact was observed in multi-object operations which are slightly more complex, as well as in single-object retrievals. In these cases the improvement suggests that these abstractions were more effective, most likely due to reduced boilerplate code. The complex operations benefited notably as well, at approximately 10.82%. These results suggest that as operational complexity increases, the relative advantage of using Firestore also increases. Overall, the results show that using Firestore can improve maintainability by about 10%. This is a substantial difference considering that significantly more complex operations could be carried out in real-world scenarios. In addition, we must also take into consideration the fact that Firestore also removes the cumbersome, tedious, and error-prone process of manually writing serialization/deserialization logic. These improvements in MI, combined with the removal of creating and upkeeping this aspect of code can be a critical determinant of the long-term evolution of a project.

7.4 Comparative feature analysis

The final aspect of the evaluation is a comparative analysis of the features in various ORM and ODM frameworks in a variety of different programming environments. Firstly, this aims to identify the important features provided by tools similar to Firestorm. Secondly, it can enable a comparison between the number of features supported by each ORM/ODM tool. Ultimately, this can situate Firestorm among the broader ecosystem of ORMs and ODMs, allowing its evaluation in terms of the features it supports.

7.4.1 Procedure

The process of identifying these tools utilized generative AI to conduct a search on the web. A list of popular ODM and ORM tools were identified by instructing ChatGPT to look online such tools using the following prompt: “Search the web to make a list of ODM and ORM tools in various frameworks”. The initial response identified a total of 16 tools across 8 different frameworks. This list of tools was expanded with four additional known ORM tools which were not included: Sequelize (Node.js, SQL), Typesaurus (Flutter, Firestore), Hibernate (Java), and Objectify (Java, Cloud datastore). This resulted to a total number of 20 tools.

In the second step, ChatGPT was tasked to identify common features across ORM and ODM tools with the following prompt: “Search the web to identify common features across ORM and ODM tools”. This led to a response containing a list of 8 features, including schema enforcement, support for asynchronous operations, and more. Like in the previous step, the list of features was expanded with 5 additional features from related works and knowledge of existing tools: transactions, real-time listeners, pagination, migration support, and aggregation query support. The final number of features explored is 13.

ChatGPT was then asked to construct a feature matrix structure of these tools and the features they support with this prompt: “Make a matrix of these tools (as rows) and the

features they support (as columns). Look on the web and make sure you find justifiable evidence online for the inclusion of such features before you confirm whether they are present in each framework or not. For each of the tools, include the resources you used to find this information as links”. This response constructed a feature matrix by using 53 resources from the web, which are listed in Appendix C. In the matrix, each supported feature is represented by a note within the cell indicating the presence of the feature.

Due to its size, the resulting matrix had to be broken into four smaller tables. In the first table, the database supported by each tool, schema enforcement support, and type safety features were included. The second table includes core capabilities such as query building, data validation, etc. whereas the third table includes data relationship and more advanced features such as listeners and pagination. The final table includes more advanced features such as aggregation, transactions, and more. Using the data provided in these tables, (a) the number of tools supporting a specific feature can be extracted, which can provide insights into which features are more popular, and (b) Firestorm’s feature support can be compared with other tools to determine its feature coverage and determine areas where further work might be needed to address additional requirements.

7.4.2 Results

The four tables containing feature support for each ORM/ODM tool are included in Appendix D. These results are summarized in Table 7.6 which presents these features sorted by frequency of tools supporting these features. The third column in the table indicates the presence of each feature in Firestorm, allowing an evaluation of the feature coverage.

Table 7.7 also shows the number of features supported by each tool, sorted in descending order. This can provide a more in-depth view of each tool’s feature coverage, allowing direct comparisons between these tools.

Feature	Number of Tools (/20)	Firestorm
Schema Enforcement	19	Yes
Type Safety	19	Yes
Maintained	19	Yes
Query Builder	17	Yes
Validation	15	Yes
Pagination	14	Yes
Relationships	14	
Hooks	12	Yes
Aggregation	12	
Async Support	10	Yes
Transactions	11	Yes
Realtime Listeners	7	Yes
Migration	2	

Table 7.6: Frequency of feature support across all tools.

Tool	Number of Features Supported (/13)
Mongoose	11
Sequelize	11
Spring Data Mongo	11
Typegoose	11
Firestorm	10
Hibernate	10
Mongoid	10
Objection.js	10
Beanie	10
cloud_firestore_odm	9
MongoDB .NET Driver	9
MongoEngine	9
Morphia	9
Objectify	9
mongo-go-driver	8
JNoSQL	7
Laravel MongoDB	6
Waterline	6
Marshmallow	5
Isar	4
Pydantic ODM	4

Table 7.7: Number of supported features per tool.

7.4.3 Discussion

The results shown in Table 7.6 reveal a high level of diversity in terms of feature support in the ORM/ODM tools studied. Feature adoption is relatively high, with most of the features being supported by the majority of the tools. The schema enforcement, type safety, and maintenance were the most popular aspects of these packages, with all but one tool supporting these features. Other strongly supported features were query building support, validation mechanisms, pagination, and relational constructs between data. Unsurprisingly, all of these features are often regarded as cornerstone requirements of such tools. Other features which are typically less supported include aggregation queries, support for asynchronous programming, transactions, and real-time listeners, whereas migration is mostly ignored.

Table 7.7 lists the number of features supported by each individual tool. 8 out of 20 (40%) of these tools had excellent (more than 75%) support for the features identified, and 13 out of 20 (65%) provided a support for the majority (at least 50%) of the features. The data in these two tables can be used to answer the fourth research question (RQ4): “*How does Firestorm compare against other ODM tools in terms of features?*”

As seen in table 7.6, Firestorm provides support for 10 out of 13 ($\approx 77\%$) of the features identified, which provides a very good level of feature coverage. This places it near the top of Table 7.7, which counts the number of features supported in each tool. Thus, in terms of feature coverage, Firestorm outperforms at least 12 other popular ORM/ODM tools, and it is only outperformed by 4. The results obtained from this aspect of the evaluation can also be very useful in determining new directions in development. For instance, next steps in development could focus on features which are currently unsupported or undersupported, such as the formation of relationships between data and support for aggregation queries.

7.5 Design strategies

Finally, the second research question can be answered based on the design decisions and strategies explained in Chapters 4 and 5. RQ2: “*Which design principles and architectural strategies enable Firestore to support both Firestore and the Realtime Database within a unified ODM framework?*”

A major difference between Firestore and most of the tools explored in the comparative feature analysis is that most of these tools focus on providing ORM/ODM support for a single type of database. While some tools such as Marshmallow provide database and framework-agnostic support for various features, these focus on data serialization and deserialization from/to platform-agnostic representations such as JSON rather than implementing specific data access APIs. An exception is Java’s Hibernate, which uses the `EntityManager` and JPA (Java Persistence API) to address the problem of managing data interactions regardless of the database used. At present, the Flutter ecosystem does not have a production-grade ORM/ODM tool that offers this level of abstraction. Firestore is built based on several architectural principles and design strategies to address this problem:

1. Multilayered modular architecture

The architecture employed by Firestore described in section 4.3 promotes the separation of concerns between different aspects of the app including data modeling, code generation, and the data access components. Firstly, this leads to a better level of maintainability across the package. More importantly however, this introduces an abstraction layer, which is present within the data access API that enables a common API to be utilized for both datastores, and any future datastore that needs to be supported.

2. Delegate pattern

An important design aspect of Firestore is the use of the Delegate pattern to encapsulate datastore-specific CRUDLE operations. The delegates implement abstract

API operations, but also further decouple code and promote abstraction and concern separation at a more granular level.

3. Facade pattern

The data access API exposes unified entry points for each datastore – the **FS** and **RDB** classes. While these are independent of each other, the use of the Facade design pattern in combination with Delegates provides a *consistent* API for both datastores and enables developers to use them simultaneously.

4. Static analysis and Code generation

Runtime reflection, a performance-degrading and unsupported feature in Flutter, has been replaced by compile-time code analysis and generation to ensure type safety and consistent serialization and deserialization. This mechanism is common across both datastores, but still handles the data type support peculiarities of each specific datastore.

5. Registry & Strategy patterns

Firestorm utilizes the Registry and Strategy patterns to dynamically map entity metadata and associated functions such as serializers and deserializers, enabling it to efficiently use statically defined data at runtime to perform class validation.

6. Builder pattern

The use of builders promotes better code structures by using the declarative programming paradigm. Using this pattern, developers can easily define complex queries that are compatible with each datastore.

7.6 Summary

This chapter has presented the evaluation of Firestorm which explored the performance, code maintainability, and feature coverage of the package. The results obtained were positive in

all aspects, and pave the way for the use of Firestorm to develop maintainable cross-platform software.

Chapter 8

Conclusion

The present thesis addressed the lack of mature, maintainable, and developer-friendly ODM tools to facilitate data management in cross-platform Flutter applications. It presented the design and development of Firestorm, an ODM and data access API which facilitates maintainable cross-platform software development, and evaluated this tool with respect to four research questions and three evaluation aspects – performance, code maintainability, and feature coverage.

8.1 Contributions

8.1.1 Objectives

The project’s objectives which were outlined in section 1.2 included – among others – the reduction of repetitive boilerplate code, the enhancement of code maintainability, the preservation of performance, and provisions for supporting multiple datastores simultaneously.

1. Declarative & type-safe data modeling

Various components in this package utilize a declarative approach to make it easier to define both data models and queries. Code analysis and generation are used to enforce type safety and a consistent but flexible schema, which reduces the likelihood

of runtime errors but also preserves the ability of software to evolve.

2. Reduction of boilerplate code

Firestore automates the logic of mapping objects to their serialized counterparts and back which significantly reduces the effort and time needed to develop and expand applications. This also prevents developers from introducing bugs within the mapping/serialization pipeline.

3. Facilitating RAD

Features such as automatic code generation and high-level abstractions in multiple layers of Firestore are considered to act as a conduit towards Rapid Application Development. Using Firestore, developers can quickly prototype applications that must utilize Firebase’s datastores for persistence with an easy-to-use, low-code, object-oriented API. This can enable the rapid prototyping of applications in educational and research settings, but also in commercial contexts where the time-to-market is more important.

4. Improved code readability, maintainability, and portability

Firestore’s object-oriented approach is inherently more readable than attribute-oriented code utilized in the native Firebase APIs, which makes it easier to read and understand. Evidence has also suggested that code maintainability is increased, especially for larger projects which typically rely on such backend services. Furthermore, the nature of Flutter as a cross-platform framework means that Firestore’s code is inherently portable.

5. A unified API for multiple datastores

Firestore provides a single, intuitive data access API which allows the development of applications that interact with multiple datastores at the same time. Even though this project focuses on Firebase’s Firestore and Realtime databases, it could be easily expanded to use the same API to support other types of databases.

8.1.2 Software requirements coverage assessment

This project has achieved its goals in several important areas and has implemented key requirements which were identified in tables 3.2 and 3.3. Tables 8.1 and 8.2, show these initial requirements with an additional column indicating their completion up to the point of delivering this thesis:

Table 8.1: Assessment of implementation for functional requirements.

ID	Requirement	Priority	Implemented
F1	Ability to define data models as annotated Dart classes	M	Yes
F2	Static type checking of data classes, for both datastores	M	Yes
F3	Verify conformance of data classes to Firestore data model	M	Yes
F4	Offer a way to register classes for use with Firestore	M	Yes
F5	Generate serialization/deserialization methods for data classes	M	Yes
F6	Utilize serialization/deserialization methods internally, without explicit developer use	M	Yes
F7	Support both Firestore and Realtime database APIs with seamless use/transition between the two	M	Yes
F8	Provide an object-oriented API that allows developers to interact with objects rather than raw data	M	Yes
F9	Enable the use of inheritance in data classes	M	Yes
F10	Enable the use of data classes within other data classes (Composition)	M	Yes
F11	Offer basic CRUD operations for Firestore	M	Yes
F12	Offer basic CRUD operations for Realtime database	M	Yes
F13	Provide ways to handle errors during operations	M	Yes
F14	Provide an asynchronous API which can be used in blocking or non-blocking mode	M	Yes
F15	Provide an API for queries and filtering for Firestore	M	Yes

Continued on next page

ID	Requirement	Priority	Implemented
F16	Provide an API for queries and filtering for Realtime database	M	Yes
F17	Offer batch operations (multi-read, write, update, delete) for Firestore	M	Yes
F18	Offer batch operations (multi-read, write, update, delete) for Realtime database	M	Yes
F19	Provide support for transactions in Firestore	M	Yes
F20	Allow the use of enums in data classes	M	Yes
F21	Allow the quick creation of randomly generated IDs for database objects	M	Yes
F22	Provide support for object-oriented real-time listener functions in Firestore	S	Yes
F23	Provide support for object-oriented real-time listener functions in Realtime database	S	Yes
F24	Enable developers to retrieve raw references to documents and work with the traditional Firestore API	S	Yes
F25	Enable developers to retrieve raw references to keys and work with the traditional Realtime database API	S	Yes
F26	Provide an easy way to enable Firestore caching through Firestore	S	Yes
F27	Provide an easy way to enable Realtime database caching through Firestore	S	Yes
F28	Provide an easy way to configure Firestore in on-line/offline mode selectively through Firestore	S	Yes
F29	Provide an easy way to shut down the Firestore connection and deallocate resources through Firestore	S	Yes
F30	Provide an easy way to shut down the Realtime database connection and deallocate resources through Firestore	S	Yes
F31	Provide support for creating subcollections in Firestore	C	Yes

Continued on next page

ID	Requirement	Priority	Implemented
F32	Provide support for creating subcollections in Realtime database	C	Yes
F33	Implement a pagination API for Firestore	C	Yes
F34	Implement a pagination API for Realtime database	C	Yes
F35	Utilize multithreading or concurrent operations where appropriate to improve performance	C	Partially
F36	Provide an easy way to use the Firestore emulator through Firestorm	W	Yes
F37	Provide an easy way to use the Realtime database emulator through Firestorm	W	Yes
F38	Provide a way to customize the name of collection for each class in Firestore and Realtime database	W	No
F39	Support for Firestore sentinels	W	No

Table 8.2: Assessment of implementation for non-functional requirements.

ID	Requirement	Priority	Implemented
NF1	Reduce boilerplate code (e.g., for serialization/deserialization)	M	Yes
NF2	Provide a better level of code maintainability compared to the traditional Firebase APIs	M	Yes
NF3	Achieve low performance overheads during database operations	M	Yes
NF4	Ensure that data mapping does not corrupt the database	M	Yes
NF5	Provide comprehensive documentation to learn the data access API and how to generate models	M	Yes
NF6	Must work on the Web	M	Partially

Continued on next page

ID	Requirement	Priority	Implemented
NF7	Must work across multiple mobile platforms including Android and iOS	M	Yes
NF8	Maintain compatibility with the security rules of Firestore and Realtime database	S	Yes
NF9	Maintain the graceful degradation of services of traditional Firebase APIs	S	Yes
NF10	Provide an extensible API that can be extended in the future	S	Yes
NF11	Must work on Desktop platforms including Windows and MacOS	S	Partially
NF12	Follow Dart and Flutter conventions and terminology to maximize adoptability	C	Yes
NF13	Support the latest stable Firebase SDKs and be interoperable with their APIs	C	Yes

8.1.3 Summary of research output

The evaluation assessed the package’s overall effectiveness with respect to the evaluation aspects identified in section 1.4. There are several outputs, which are used to answer the four research questions posed in section 1.3.

The maintainability index analysis has shown code maintainability improvements across all of the tested operations – even among relatively simple CRUDLE operations where the code is already simple. While not covering all of the operations, the sample of operations tested presents a representative sample of various types of operations typically used by developers to manage data. Thus, this thesis claims that code written using Firestorm has the potential of improving code maintainability by an average of 10% compared to using the native Firebase

Flutter database SDKs.

Another research output of this thesis is presented in section 7.5, which presents a list of 6 design strategies used to enable Firestore to support both Firestore and the Realtime database within a single ODM tool. Firestore’s ability to handle simultaneous operations across both datastores provides proof that these design strategies have successfully enabled Firestore to meet its objectives. Such design strategies not only simplify development but also provide the underpinnings for seamless interoperability between datastores.

Performance-wise, the results obtained from running isolated data access API code tests show that Firestore not only stays within the acceptable 10% processing time increase threshold, but surprisingly outperforms the native APIs in several operations. In some of these operations, significant performance gains were also observed, highlighting the success of Firestore as a performance-sensitive data access API that allows efficient operations despite the added layers of processing needed to enable ODM.

In terms of feature coverage, the comparative analysis conducted indicates that Firestore implements the majority (77%) of the features typically expected in a mature ORM/ODM framework, and also exceeds or ties most (80%) of the identified tools in terms of feature coverage.

8.2 Limitations and future work

Despite these very promising results, there are several limitations in both the development and testing of Firestore, as well as the methods and processes used to evaluate it. These are discussed below, along with plans to mitigate these limitations through future work.

8.2.1 Development and Testing

Firstly, several constraints impacted the scope and completeness of this framework. Time constraints imposed by the project’s schedule meant that several features which might be

useful to developers such as data relationships and aggregation queries could not be implemented. While the time constraints did not allow for the implementation of such features, the package’s underlying architecture enables the expansion of its APIs to easily implement these capabilities in the future.

Secondly, the same time constraints greatly limited the amount of coverage in terms of testing. Despite testing and verifying Firestorm within an Android environment, there is limited proof that this package is compatible across other platforms such as iOS, Web, and Windows. While an analysis of the internal dependencies shows support for these platforms, further platform compatibility testing is required to ensure that various platform-specific issues do not arise and remain undetected.

Thirdly, while Firestorm is designed to enforce type safety and validation operations at runtime, there could be further issues which might have escaped testing. For instance, runtime configuration errors or particular edge cases during serialization/deserialization could still introduce hard-to-detect bugs in the package which would only be detected through its usage (in time), or through exhaustive testing. This is typical for cases where automated unit tests cannot be performed due to running compile-time tasks. Therefore, future work should focus on maintaining the package by tracking issues that arise and fixing the underlying causes to improve its quality, and by improving the test coverage.

8.2.2 Research approach

Despite successfully addressing several research questions, the research approach used may limit the generalizability of the results presented in this thesis.

The package’s performance benchmarking was conducted in a controlled environment to ensure the reproducibility and reliability of the results. This entailed the use of specific hardware and configurations, consistent data and operations, and the use of identical versions of software tools. However, while this approach ensures reproducibility, it also limits the generalizability of the results. For instance, real-world deployments involving the

actual datastores (not emulators), diverse network conditions, varying device capabilities, and production-scale data volumes could lead to dramatically different performance characteristics. Future work can therefore focus on simulating real-world, production-scale data requirements, more diverse conditions in varying usage contexts, as well as measuring the memory overheads.

Furthermore, the maintainability analysis based on the Maintainability Index metric uses pre-defined snippets of code from the native APIs and Firestore. Despite the fact that MI is a composite metric which captures three different dimensions of maintainability, it still does not capture all dimensions. For instance, MI does not provide insights with regards to developer perceptions, the learnability of the code, or the long-term maintainability under use by large groups of developers. Importantly, MI measurements in native API code snippets utilized Firestore’s serialization tools, which may have provided a slight edge towards the measurements obtained for these snippets. Future work can thus focus on additional maintainability aspects which were not addressed in this thesis, starting with recording and addressing developer feedback and perceptions through a qualitative study, but also ensuring more fairness during code analysis.

The comparative analysis of feature coverage relied on publicly available information obtained from the web with the help of a generative AI tool. While the documentation and online resources identified were verified as authentic, there are concerns over the accuracy of the results produced by an AI tool, especially when it comes to identifying the right features and tools. While this was mitigated to a limited extent by manually providing additional features and known ORM/ODM tools, a more exhaustive search could be necessary to identify tools, features, and resources which may have been overlooked. In the future, a more strategic approach could be followed to ensure that the data retrieved is more precise and covers other important aspects of feature coverage.

8.3 Epilogue

Despite these limitations, Firestorm has the potential to significantly improve the maintainability of cross-platform software which incorporates data-driven operations. The project has completed most of the requirements identified, and the thesis has presented results which have demonstrated significant advantages in terms of the evaluation aspects. This package is now publicly available and open-source, and the author is hopeful that it may not only improve the process of developing cross-platform software, but also inspire further research in this domain.

Appendix A

Unit test cases

Table A.1: Unit Test Cases

Function ID	Test ID	Test	Preconditions	Input	Expected Output	Postconditions
1	a	Class or its super-classes contain ID field of type String	Class elements detected in code. ClassElement has been constructed.	ClassElement with no id field	Empty Valid- ClassHolder	No code is generated for class.
1	b	Class or its super-classes contain ID field of type String	Class elements detected in code. ClassElement has been constructed.	ClassElement with id field of type int (other than String)	Empty Valid- ClassHolder	No code is generated for class.
1	c	Class or its super-classes contain ID field of type String	Class elements detected in code. ClassElement has been constructed.	ClassElement with id field of type String	ValidClassHolder with classElement	Code generated for class.

Continued on next page

Function ID	Test ID	Test	Preconditions	Input	Expected Output	Postconditions
1	d	Class or its super-classes contain ID field of type String	Class elements detected in code. ClassElement has been constructed.	ClassElement superclass containing id field & ClassElement not containing id field	ValidClassHolder with classElement	Code generated for class.
1	e	Class or its super-classes contain ID field of type String	Class elements detected in code. ClassElement has been constructed.	ClassElement superclass containing id field & ClassElement containing id field	Compile-time error (Dart)	No compilation
2	a	Class attributes are valid types	Class elements detected in code. ClassElement has been constructed.	ClassElement with 1 Dart core type	ValidClassHolder with classElement	Code generated for class.
2	b	Class attributes are valid types	Class elements detected in code. ClassElement has been constructed.	ClassElement with 1 Dart core and 1 Firestore SDK valid type	ValidClassHolder with classElement	Code generated for class.

Continued on next page

Function ID	Test ID	Test	Preconditions	Input	Expected Output	Postconditions
2	c	Class attributes are valid types	Class elements detected in code. ClassElement has been constructed.	ClassElement with 1 Dart core, 1 Firestore SDK valid type, and 1 composite type	ValidClassHolder with classElement	Code generated for class.
2	d	Class attributes are valid types	Class elements detected in code. ClassElement has been constructed.	ClassElement with 1 Dart core, 1 Firestore SDK valid type, 1 composite type, 1 enum type	ValidClassHolder with classElement	Code generated for class.
2	e	Class attributes are valid types	Class elements detected in code. ClassElement has been constructed.	ClassElement with 1 Dart core, 1 Firestore SDK valid type, 1 composite type, 1 enum type, 1 invalid composite type	Empty ValidClassHolder	No code is generated for class.

Continued on next page

Function ID	Test ID	Test	Preconditions	Input	Expected Output	Postconditions
3	a	Extension methods generated correctly	ValidClassHolder created, contains ClassElement	ClassElement with valid attributes and a constructor with positional params only	Success	Code generated for class.
3	b	Extension methods generated correctly	ValidClassHolder created, contains ClassElement	ClassElement with valid attributes and a constructor with positional and named params	Success	Code generated for class.
3	c	Extension methods generated correctly	ValidClassHolder created, contains ClassElement	ClassElement with valid attributes and a constructor with positional, named, and optional params	Success	Code generated for class.

Continued on next page

Function ID	Test ID	Test	Preconditions	Input	Expected Output	Postconditions
3	d	Extension methods generated correctly	ValidClassHolder created, contains ClassElement	ClassElement with valid attributes and a constructor with positional and optional params	Success	Code generated for class.

Appendix B

Integration test cases

Table B.1: Integration test cases

Function ID	Test ID	Test	Input	Expected Output
1	a	Test FS.create.one()	Object	result != null
2	a	Test FS.create.many()	List of objects	result.length == objects.length
3	a	Test FS.get.one()	String id (valid)	result != null
3	b	Test FS.get.one()	String id (invalid)	result == null
4	a	Test FS.get.many()	List of IDs	result.length == objects.length; for all objects: result.contains(object)
5	a	Test FS.list.ofClass()	Type	result.length == objects.length

Continued on next page

Function ID	Test ID	Test	Input	Expected Output
6	b	Test FS.list.filter()	Type, query clause (height <= 1.70, limit 5, order by height)	result.length <= 5; for all results: element.height <= 1.70; for all results until previous to last: result[i].height != result[i+1].height
7	a	Test paginate()	Type, limit: 3, 3 calls, 10 items	currentPage.items.length <= 3, over 3 pages
8	a	Test FS.batch.run()	List of objects	result.length == objects.length
9	a	Test FS.transaction.run()	List of objects, updated firstname attribute	result.length == objects.length; for all objects: object.firstname == "updated"

Appendix C

Feature matrix resources

C.1 Mongoose

- Mongoose Change Streams Documentation: <https://mongoosejs.com/docs/change-streams.html>
- Mongoose Validation Documentation: <https://mongoosejs.com/docs/validation.html>
- Wanago Blog – MongoDB Transactions with Mongoose: <https://wanago.io/2021/09/06/api-nestjs-transactions-mongodb-mongoose/>
- GitHub – Typegoose Pagination Plugin: <https://github.com/tecmie/typegoose-pagination-plugin>

C.2 Typegoose

- Typegoose Model Options Decorators: <https://typegoose.github.io/typegoose/docs/api/decorators/model-options/>
- Typegoose Property Decorators: <https://typegoose.github.io/typegoose/docs/>

`api/decorators/prop/`

- GitHub – Typegoose Pagination Plugin: <https://github.com/tecmie/typegoose-pagination-plugin>

C.3 Sequelize

- Sequelize Hooks Documentation: <https://sequelize.org/docs/v6/other-topics/hooks/>
- Sequelize Transactions Documentation: <https://sequelize.org/docs/v6/other-topics/transactions/>
- Sequelize Querying Documentation: <https://sequelize.org/docs/v6/core-concepts/model-querying-finders/>

C.4 Objectify

- Objectify GitHub Repository: <https://github.com/objectify/objectify>
- Stack Overflow – Objectify Pagination Discussion: <https://stackoverflow.com/questions/35250643/pagination-in-google-cloud-endpoints-datastore-objectify>

C.5 Beanie

- Beanie Official Documentation: <https://beanie-odm.dev/>
- Talk Python Podcast – Beanie ODM Introduction: <https://talkpython.fm/episodes/show/349/meet-beanie-a-mongodb-odm-pydantic>

- Amrit Panta Blog – Beanie Guide: <https://blog.amritpanta.com.np/2024/11/17/learn-beanie-a-beginners-guide-to-async-python-odm-with-fastapi/>
- MongoDB Developer Article – Aggregation with Beanie: <https://www.mongodb.com/developer/languages/python/beanie-odm-fastapi-cocktails/>

C.6 MongoEngine

- MongoEngine Validation Guide: <https://docs.mongoengine.org/guide/validation.html>
- MongoEngine Aggregation API Reference: <https://docs.mongoengine.org/apireference.html#mongoengine.queryset.QuerySet.aggregate>

C.7 Spring Data MongoDB

- Spring Data MongoDB Change Streams Documentation: <https://docs.spring.io/spring-data/mongodb/reference/mongodb/change-streams.html>
- Spring Data MongoDB Aggregation Framework: <https://docs.spring.io/spring-data/mongodb/docs/current-SNAPSHOT/reference/html/#mongo.aggregation>
- Spring Data MongoDB Transactions: <https://docs.spring.io/spring-data/mongodb/docs/current-SNAPSHOT/reference/html/#transactions>

C.8 JNoSQL

- JNoSQL GitHub Repository: <https://github.com/eclipse/jnosql>
- JNoSQL Specification: <https://www.jnosql.org/spec/>

- InfoQ Article – Leveraging JNoSQL: <https://www.infoq.com/articles/leveraging-jnosql-1-0/>
- Open Liberty Blog – Jakarta NoSQL in Action: <https://openliberty.io/blog/2024/05/13/jakarta-nosql-in-action-meet-jnopo-game.html>

C.9 MongoDB .NET Driver

- MongoDB C# Driver Change Streams Documentation: <https://www.mongodb.com/docs/drivers/csharp/current/fundamentals/crud/read-operations/change-streams/>
- MongoDB C# Driver Transactions Documentation: <https://www.mongodb.com/docs/drivers/csharp/current/fundamentals/transactions/>
- MongoDB Community Forum – Pagination in C# Driver: <https://www.mongodb.com/community/forums/t/how-to-do-pagination-with-c-driver/280921>

C.10 Mongoid

- Mongoid Official Documentation: <https://docs.mongodb.com/mongoid/master/>
- MongoDB Ruby Driver Change Streams Documentation: <https://www.mongodb.com/docs/ruby-driver/current/read/change-streams/>

C.11 Marshmallow

- Marshmallow Documentation: <https://marshmallow.readthedocs.io/en/stable/>

C.12 Objection.js

- Objection.js Official Documentation: <https://vincit.github.io/objection.js/>
- Objection.js GitHub Repository: <https://github.com/Vincit/objection.js>

C.13 Morphia

- Morphia Official Documentation: <https://morphia.dev/>
- Morphia GitHub Repository: <https://github.com/MorphiaOrg/morphia>

C.14 Laravel MongoDB (Jenssegers)

- Laravel MongoDB GitHub Repository: <https://github.com/jenssegers/laravel-mongodb>
- Laravel MongoDB Documentation: <https://jenssegers.com/projects/laravel-mongodb>
- Laravel Eloquent Documentation: <https://laravel.com/docs/eloquent>

C.15 Waterline (Sails.js ORM)

- Waterline Official Documentation: <https://sailsjs.com/documentation/concepts/models-and-orm>
- Waterline GitHub Repository: <https://github.com/balderdashy/waterline>

C.16 Pydantic ODM

- Pydantic Official Documentation: <https://docs.pydantic.dev/>

C.17 Isar

- Isar Official Documentation: <https://isar.dev/>
- Isar GitHub Repository: <https://github.com/isar/isar>

C.18 Typesaurus

- Typesaurus GitHub Repository: <https://github.com/typesaurus/typesaurus>
- Typesaurus Documentation: <https://typesaurus.com/>

C.19 mgo/bson

- mgo (Labix) Documentation (Archived): <https://godoc.org/gopkg.in/mgo.v2>
- mgo GitHub Repository (Community Fork): <https://github.com/globalsign/mgo>

C.20 mongo-go-driver

- MongoDB Go Driver Documentation: <https://www.mongodb.com/docs/drivers/go/current/>
- MongoDB Go Driver GitHub Repository: <https://github.com/mongodb/mongo-go-driver>

C.21 bongo

- bongo GitHub Repository: <https://github.com/go-bongo/bongo>

C.22 NoRM

- NoRM GitHub Repository (Archived): <https://github.com/atheken/NoRM>

C.23 Hibernate

- Official Hibernate ORM documentation: <https://hibernate.org/orm/documentation/>
- Hibernate user guide: https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html

Appendix D

ODM Feature tables

D.1 Databases, type safety, schemas

Table D.1: ODM tool features – Database, type safety, and schema enforcement

Tool	Database	Schema Enforcement	Type Safety
Beanie	MongoDB	Yes	Yes
cloud_firestore_odm	Firestore	Yes	Yes
Isar	Local	Yes	Yes
JNoSQL	NoSQL	Yes	Java
Laravel MongoDB	MongoDB	Yes	PHP
Marshmallow	Any	Yes	Yes
MongoDB .NET Driver	MongoDB	Partial	C#
MongoEngine	MongoDB	Yes	Python
Mongoid	MongoDB	Yes	Ruby
Mongoose	MongoDB	Yes	Partial
Morphia	MongoDB	Partial	Java
mongo-go-driver	MongoDB		Go
Objectify	Datastore	Partial	Java
Objection.js	SQL	Yes	Partial
Pydantic ODM	MongoDB	Yes	Yes
Sequelize	SQL	Yes	Partial

Continued on next page

Tool	Database	Schema Enforcement	Type Safety
Spring Data Mongo	MongoDB	Yes	Java
Typegoose	MongoDB	Yes	Yes
Waterline	MongoDB+	Yes	Partial
Hibernate	Any	Yes	Yes

D.2 Queries, asynchronous operations, hooks, and validation

Table D.2: ODM tool features – Queries, Async support, Hooks, and Validation

Tool	Query Builder	Async Support	Hooks	Validation
Beanie	Yes	Yes	Yes	Yes
cloud_firestore_odm	Yes	Yes		Yes
Isar				
JNoSQL	Yes		Yes	Yes
Laravel MongoDB	Yes			Yes
Marshmallow			Yes	Yes
MongoDB .NET Driver	Yes	Yes		
MongoEngine	Yes		Yes	Yes
Mongoid	Yes		Yes	Yes
Mongoose	Yes	Yes	Yes	Yes
Morphia	Yes		Yes	
mongo-go-driver	Yes	Yes		
Objectify	Yes	Yes		Yes
Objection.js	Yes	Yes	Yes	Yes
Pydantic ODM				
Sequelize	Yes	Yes	Yes	Yes
Spring Data Mongo	Yes	Yes	Yes	Yes
Typegoose	Yes	Yes	Yes	Yes
Waterline	Yes			Yes

Continued on next page

Tool	Query Builder	Async Support	Hooks	Validation
Hibernate	Yes		Yes	Yes

D.3 Relationships, realtime listeners, and pagination

Table D.3: ODM tool features – Relationships, Realtime listeners, and Pagination

Tool	Relationships	Realtime Listeners	Pagination
Beanie	Yes		Yes
cloud_firestore_odm	Partial	Yes	Yes
Isar			
JNoSQL	Yes		
Laravel MongoDB	Yes		
Marshmallow			
MongoDB .NET Driver		Yes	Yes
MongoEngine	Yes		Yes
Mongoid	Yes	Yes	Yes
Mongoose	Yes	Yes	Yes
Morphia	Yes		Yes
mongo-go-driver		Yes	Yes
Objectify	Partial		Yes
Objection.js	Yes		Yes
Pydantic ODM			
Sequelize	Yes		Yes
Spring Data Mongo	Yes	Yes	Yes
Typegoose	Yes	Yes	Yes
Waterline	Yes		
Hibernate	Yes		Yes

D.4 Aggregation, transactions, migration, and maintenance

Table D.4: ODM tool features – Aggregation, Transactions, Migration, and Maintenance

Tool	Aggregation	Transactions	Migration	Maintained
Beanie	Yes			Yes
cloud_firestore_odm		Yes		Yes
Isar				Yes
JNoSQL				Yes
Laravel MongoDB				Yes
Marshmallow				Yes
MongoDB .NET Driver	Yes	Yes		Yes
MongoEngine	Yes			Yes
Mongoid	Yes			Yes
Mongoose	Yes	Yes		Yes
Morphia	Yes	Yes		Yes
mongo-go-driver	Yes	Yes		Yes
Objectify		Yes	Yes	Yes
Objection.js	Yes	Yes		Yes
Pydantic ODM				Yes
Sequelize	Yes	Yes	Yes	Yes
Spring Data Mongo	Yes	Yes		Yes
Typegoose	Yes	Yes		Yes
Waterline				
Hibernate	Yes			Yes

Bibliography

- Antoy, S. and Hanus, M. (2005), Declarative programming with function patterns, *in* ‘International Symposium on Logic-Based Program Synthesis and Transformation’, Springer, pp. 6–22.
- Barker, S. (2013), ‘Building node.js and mongodb applications with mongoose’, *IBM DeveloperWorks* . <https://developer.ibm.com/articles/wa-nodejs-mongoose-app/>.
- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, W. and Vogels, W. (2007), Dynamo: Amazon’s highly available key-value store, *in* ‘Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles’, ACM, pp. 205–220.
- Gajendran, A. and Ghosh, S. (2017), ‘Scalability and performance analysis of firebase real-time database’, *International Journal of Computer Applications* **162**(7), 1–5.
- Halvorsen, H.-P. (2014), ‘Introduction to database systems’, *Telemark University College*, dated Mar **3**, 40.
- Han, J., Haihong, E., Le, G. and Du, J. (2011), ‘Survey on nosql databases’, *IEEE International Conference on Pervasive Computing and Applications* pp. 363–366.
- Hare, E. and Kaplan, A. (2017), ‘Designing modular software: a case study in introductory statistics’, *Journal of Computational and Graphical Statistics* **26**(3), 493–500.

- Heričko, T. and Šumak, B. (2023), ‘Exploring maintainability index variants for software maintainability measurement in object-oriented systems’, *Applied Sciences* **13**(5), 2972.
- Hevner, A. R., March, S. T., Park, J. and Ram, S. (2004), ‘Design science in information systems research’, *MIS Quarterly* **28**(1), 75–105.
- Kamal, S. H., Elazhary, H. H. and Hassanein, E. E. (2019), ‘A qualitative comparison of nosql data stores’, *International Journal of Advanced Computer Science and Applications* **10**(2).
- Karanikiotis, T. and Symeonidis, A. L. (2024), ‘Towards understanding the impact of code modifications on software quality metrics’, *arXiv preprint arXiv:2404.03953*.
- Kasenides, N. (2023), Models, methods, and tools for developing MMOG backends on commodity clouds, PhD thesis, UCLan Cyprus.
- Kasenides, N. and Paspallis, N. (2022), ‘Athlos: A framework for developing scalable mmog backends on commodity clouds’, *Software* **1**(1), 107–145.
- Kasenides, N., Piki, A. and Paspallis, N. (2023), Exploring the user experience and effectiveness of mobile game-based learning in higher education, *in* ‘International Conference on Human-Computer Interaction’, Springer, pp. 72–91.
- Kautz, K., Johanson, T. H., Uldahl, A. et al. (2014), ‘The perceived impact of the agile development and project management method scrum on information systems and software development productivity’, *Australasian Journal of Information Systems* **18**(3).
- Kravchenko, T., Bogdanova, T. and Shevgunov, T. (2022), Ranking requirements using moscow methodology in practice, *in* ‘Computer Science On-line Conference’, Springer, pp. 188–199.
- Lakshman, A. and Malik, P. (2010), ‘Cassandra: a decentralized structured storage system’, *ACM SIGOPS Operating Systems Review* **44**(2), 35–40.

- Lee, E. A., Bateni, S., Lin, S., Lohstroh, M. and Menard, C. (2021), ‘Quantifying and generalizing the cap theorem’, *arXiv preprint arXiv:2109.07771* .
- Liao, Z., Zhang, R., He, S., Zeng, D., Wang, J. and Kim, H.-J. (2019), ‘Deep learning-based data storage for low latency in data center networks’, *IEEE Access* **7**, 26411–26417.
- Liu, P.-J., Li, C.-P. and Chen, H. (2024), ‘Enhancing storage efficiency and performance: A survey of data partitioning techniques’, *Journal of Computer Science and Technology* **39**(2), 346–368.
- Maier, D. (1983), *The theory of relational databases*, Vol. 11, Computer science press Rockville.
- Mathiassen, L., Munk-Madsen, A., Nielsen, P. A. and Stage, J. (2000), *Object-oriented analysis & design*, Vol. 25, Citeseer.
- Mayvan, B. B., Rasoolzadegan, A. and Yazdi, Z. G. (2017), ‘The state of the art on design patterns: A systematic mapping of the literature’, *Journal of Systems and Software* **125**, 93–118.
- Moniruzzaman, A. and Hossain, S. A. (2013), ‘Nosql database: New era of databases for big data analytics-classification, characteristics and comparison’, *International Journal of Database Theory and Application* **6**(4), 1–14.
- Moroney, L. (2017), *The Definitive Guide to Firebase: Build Android Apps on Google’s Mobile Platform*, Apress.
- Pavlov, M. et al. (2016), ‘Serverless computing: Economic and architectural impact’, *IEEE Cloud Computing* **3**(5), 24–30.
- Pitoura, E. and Samaras, G. (2012), *Data management for mobile computing*, Vol. 10, Springer Science & Business Media.

- Pokorny, J. (2013), ‘Nosql databases: a step to database scalability in web environment’, *International Journal of Web Information Systems* **9**(1), 69–82.
- Ponnusamy, S. and Gupta, P. (2024), ‘Scalable data partitioning techniques for distributed data processing in cloud environments: A review’, *IEEE Access* **12**, 26735–26746.
- pub.dev (2025), ‘firestore_odm — Flutter package — pub.dev’, https://pub.dev/packages/firestore_odm. [Accessed 02-07-2025].
- Ravindran, A. and George, A. (2018), An edge datastore architecture for {Latency-Critical} distributed machine vision applications, *in* ‘USENIX workshop on hot topics in edge computing (HotEdge 18)’.
- Reniers, V., Van Landuyt, D. and Joosen, W. (2017), Object-nosql database mappers: A benchmark study on the performance overhead, *in* ‘Proceedings of the 12th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)’, SciTePress, pp. 141–149.
- Riesener, M., Doelle, C., Perau, S., Lossie, P. and Schuh, G. (2021), ‘Methodology for iterative system modeling in agile product development’, *Procedia Cirp* **100**, 439–444.
- Rodriguez, M. A. (2015), ‘The graph traversal pattern’, *Communications of the ACM* **58**(3), 128–136.
- Sadalage, P. J. and Fowler, M. (2012), *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, Addison-Wesley.
- Schroeder, D., Paspallis, N., Kasenides, N., Chatfield, K., Seedall, C. and Singh, M. (2025), ‘The prepared case study app for deep reflection on research ethics challenges during crises’, *Nature Medicine* pp. 1–2.
- Shethiya, A. S. (2025), ‘Load balancing and database sharding strategies in sql server for large-scale web applications’, *Journal of Selected Topics in Academic Research* **1**(1).

- Stonebraker, M. (2010), ‘Sql databases v. nosql databases’, *Communications of the ACM* **53**(4), 10–11.
- Vom Brocke, J., Hevner, A. and Maedche, A. (2020), ‘Introduction to design science research’, *Design science research. Cases* pp. 1–13.
- Yazid, Y., Ez-Zazi, I., Guerrero-González, A., El Oualkadi, A. and Arioua, M. (2021), ‘Uav-enabled mobile edge-computing for iot based on ai: A comprehensive review’, *Drones* **5**(4), 148.
- Yousuff, M., Anusha, V., Vijayashree, J. and Jayashree, J. (2023), The role of artificial intelligence for intelligent mobile apps, *in* ‘Emerging Technologies for Sustainable and Smart Energy’, CRC Press, pp. 185–205.
- Zhang, C., Peng, J., Xu, C., Xu, Q. and Yang, C. (2025), ‘Mitigating the impedance mismatch between prediction query execution and database engine’, *Proceedings of the ACM on Management of Data* **3**(3), 1–28.